

CS5351 Software Engineering 2025-2026 Semester A

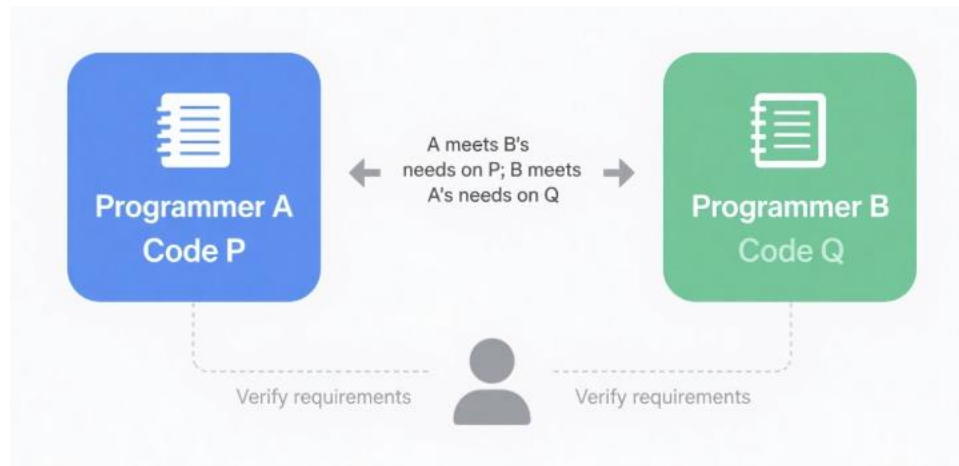
Software Process Model

Prof. Zhi-An Huang

Email: `huang.za@cityu-dg.edu.cn`

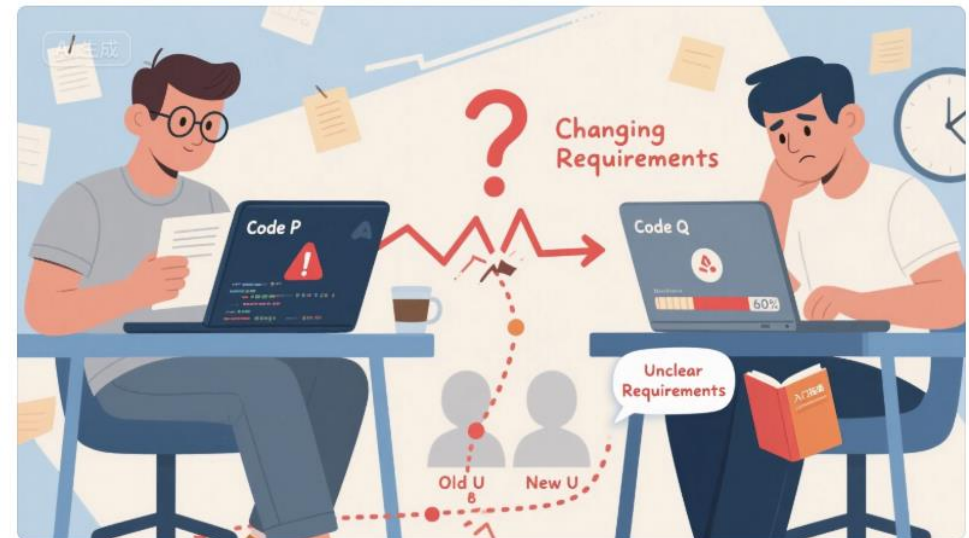
Motivation

- ◆ A group of *strong and committed* programmers (say $G = \{\text{Programmer } A, \text{Programmer } B\}$) can hack on a code project to deliver an application to their users.
 - A knows the code P and its responsibility extremely well
 - B knows the code Q and its responsibility extremely well
 - A can deliver the code P , and B can deliver the code Q
 - A meets B 's needs on P ; B meets A 's needs on Q
 - A and B can check with users U to get and verify the exact user requirements whenever necessary
- ◆ A perfect case.



Motivation (in Reality)

- ◆ A group of **average, sometimes emotional**, programmers (say $G = \{\text{Programmer } A, \text{Programmer } B\}$) can hack on a code project to deliver an application to their users.
 - A knows the code P and its responsibility **gradually**
 - B knows the code Q and its responsibility **gradually**
 - A can deliver a **buggy** P with **incomplete** functionality, and B can deliver a **buggy** Q with **incomplete** functionality
 - A **may not know** B 's **exact** needs on P ; B **may not know** A 's **exact** needs on Q . As B only knows Q gradually, B 's requirement on P changes over time.
 - A and B can check with users U to get and verify the exact user requirements **sometimes**; but an **old U resigns**, and a **replacement U comes!** Some U s are **inexperience** in their business domain.



Motivation (For Larger Project In Reality)

Suppose in Month 1, the group $G = \{\text{Programmer } A, \text{Programmer } B\}$

- ◆ Month 2, $G = \{\text{Programmer } B\}$, A resigned.
- ◆ Month 3, $G = \{\text{Programmer } A1, \text{Programmer } B\}$, $A1$ replaces A 's position.
- ◆ Month 4, $G = \{\text{Programmer } A1\}$, B resigned.
- ◆ Month 5, $G = \{\text{Programmer } A1, \text{Programmer } B1\}$, $B1$ replaces B 's position.
- ◆ Month 6, $G = \{\text{Programmer } B1\}$, $A1$ resigned.
- ◆ Month 7, $G = \{\text{Programmer } A2, \text{Programmer } B1\}$, $A2$ replaces $A1$'s position.

Analysis

- ◆ At Month 1, A and B meet with U to get user requirements
- ◆ At Month 7, $A2$ gets the needs on code P via $A \Rightarrow B \Rightarrow A1 \Rightarrow B1 \Rightarrow A2$
 - To what extent can $A2$ get what to be done on P ?
 - Can $A2$ clarify the needs of P with $B1$? Probably incomplete, but better than none

Motivation (For Larger Project In Reality)

- ◆ A2 wishes to know
 - The requirement (denoted as R) that A collected from U (months ago!)
 - The latest changes in R before A2 takes up the position
 - The set of functions $\{f_1, \dots, f_n\}$ of P that has been identified and designed but not been implemented (in full) with respect to the latest requirements
 - The feedback (from B or U) on the implemented functions $\{g_1, \dots, g_m\}$ of P with respect to the latest requirements
 - Any known and critical issues in P but not completely fixed
- ◆ B2 wishes to minimize his/her workload to pass on the information of P to A2
 - Actually, A1 also wishes to minimize his/her workload to pass information of Q to B2
- ◆ U does not want A-A2 and B-B1 to affect U's daily jobs.
 - Entertaining A/B is a side business only.
 - Can you expect an estate agent as U to meet with A-A2 and B-B1 instead of meeting with his/her clients (thus, no commissions are received, or a lower salary is received)?
- ◆ Everyone wishes to get quick references and move forwards
- ◆ When a project is large, no one knows all the details of whole project.
 - Waiting for clear instruction and clear task specification before doing the task is unrealistic.

Needs

- ◆ The software project in the previous few slides needs a strategy to control the order of project tasks to
 - to reduce production overheads,
 - improve information passing efficiency,
 - give tractability from user expectation to delivery,
 - predict and track task/project completion and outstanding issues,
 - improve code quality, and
- ◆ Otherwise....[next slide]

Bugs and Delays are Ubiquitous:

COMPUTERWORLD

UNITED STATES ▼

IDG TECH(TALK) COMMUNITY

WINDOWS

MOBILE

OFFICE SOFTWARE

APPLE

SHARK TANK

EVENTS

INSIDER

Home > Vertical Industries > Retail Industry

SLIDESHOW

Top software failures in recent history

The biggest software failures in recent history including ransomware attacks, IT outages and data leakages that have affected some of the biggest companies and millions of customers around the world

By Computerworld UK staff, Computerworld | Feb 17, 2020 6:20 am PST

February 2020: Heathrow disruption

More than 100 flights to and from London's Heathrow airport were disrupted on Sunday 16 February, 2020, after it was hit by technical issues affecting departure boards and check-in systems, leaving passengers with little information about their flights and limiting the use of electronic tickets.

August 2019: British Airways (again)

British Airways was struck by yet another IT glitch in August 2019, when system failures caused more than 100 flights to be cancelled and more than 200 others to be delayed.

Facebook, Instagram and WhatsApp

In the first week of July 2019, users across the globe found themselves unable to load photos in the Facebook News Feed, view stories on Instagram, or send messages in WhatsApp. Although Facebook didn't detail the reason for the outage, it did release a

Bitcoin Unlimited

Bitcoin Unlimited suffered a serious memory leak which caused several nodes to fall from 800 to about 300. This is almost 70 percent of the nodes run by Bitcoin Unlimited at the time.

HSBC suffers major outage

Less than a week into 2016, HSBC became the first bank to suffer a major IT outage. Millions of the bank's customers were unable to access online accounts. Services only returned to normal after a two-day outage.

Bugs and Delays are Ubiquitous: Airport Chaos and System Upgrade

◆ Hong Kong Airport

You are here: [silicon.com](#) > [Hardware](#) > [PDAs](#)

PDAs

Hong Kong airport systems grind to a halt

By Polly Raymond

Published: 9 July 1998 09:37 BST

Computer bugs have brought Hong Kong's airport cargo handling system to its knees.

Show related articles

The \$20bn (£12.2bn) airport was only opened on Monday but has quickly stumbled upon system problems, which have caused 24 hour delays for some travellers.

A spokeswoman for the Hong Kong Embassy said: "It's only a minor problem" but was unable to confirm whether systems have returned to normal.

The bugs are in the cargo IT systems and baggage handling operations. Cargo operations have been

Hong Kong's air traffic controllers left flying blind for six minutes as HK\$1.56 billion system malfunctions again – but authorities insist safety was not affected

Civil Aviation Department recognises 'importance of incident' as lawmaker Jeremy Tam questions delay in alerting public about the failure

PUBLISHED: Wednesday, 11 August, 2015, 11:45pm
UPDATED: Wednesday, 11 August, 2015, 11:21pm

COMMENTS: 5

Hong Kong's air traffic control system suffers another glitch

Back-up system deployed for first time as operators lose information on positioning and altitude of planes

PUBLISHED: Saturday 08 April, 2017, 8:00pm
UPDATED: Saturday 08 April, 2017, 10:13pm

COMMENTS: 11



<https://www.scmp.com>

UK Airport Chaos Due To Software Glitch

By Spyros Georgilidakis | May 27, 2023



Passengers arriving at more than one UK airport faced severe delays because of software problems with air control systems.

MTR

Hong Kong / Hong Kong economy

Software error to blame for two-hour Hong Kong MTR stoppage that left tens of thousands stranded

Transport operator says problematic code has been fixed, and improved recovery procedures put in place



Danny Mok

+ FOLLOW

Published: 8:50am, 13 Mar, 2018

Why you can trust SCMP



SIGNALLING BUG DELAYS MTR'S SHA TIN-CENTRAL EXTENSION MILESTONE

BY JAMES OCKENDEN ON SEPTEMBER 12, 2020

HONG KONG

Signal system supplier confirms software issue after Central train crash, MTR Corp says



by KRIS CHENG

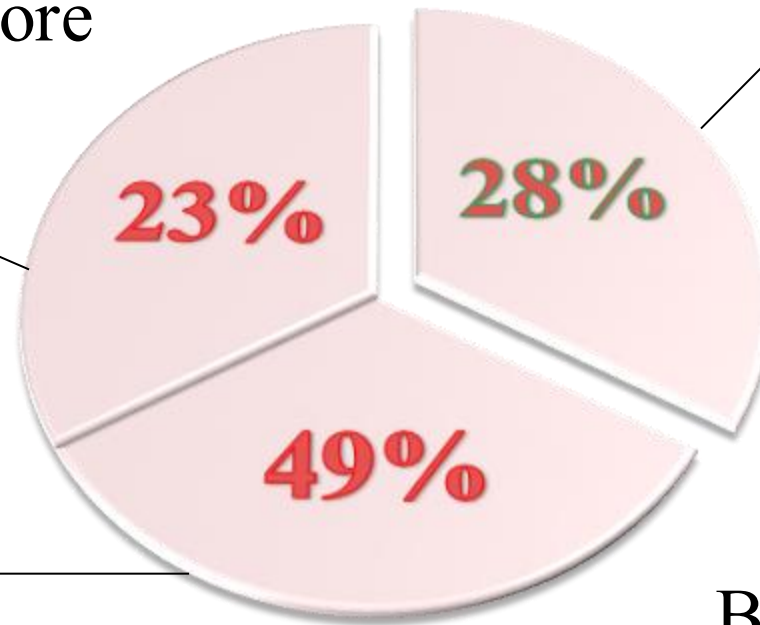
19:36, 18 MARCH 2019 Updated 22:58, 31 MARCH 2020



Facts (quite some years ago): **Software Development *Project***

cancelled before
completion.

cost 89%
more than
their original
budget



completed **on**
time and
within
budget.

But, only **42%** of the
original requirements
can be implemented.

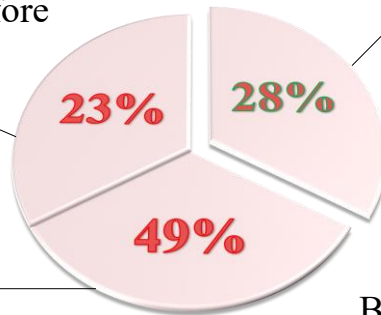
Problems

Nothing can be useful or reused from such cancelled projects?

Facts:
Software Development Project

cancelled before completion.

cost 89% more than their original budget



completed **on time and within budget.**

But, only **42%** of the original requirements can be implemented.

How to identify a subset of requirements, which are consistent among them, and track their development

Need to prioritize the requirements to guide the development effort

How to streamline the activities better to reduce cost?

More Recent Findings

- ◆ Standish Group's Annual CHAOS 2020 report
 - 66% of projects end in partial/total failure.
 - 31% canceled
 - 53% with performance degrade
 - Large project: success rate 10%
 - Small project: success rate 90%
- ◆ McKinsey in 2020
 - 17% of large projects go badly
- ◆ BCG (2020)
 - 70% of digital transformation efforts fall short of meeting targets.

Current Situation

Project Failure in 2018

FEATURE

IT project success rates finally improving

After years of stagnating IT project success rates, a new survey from PMI shows that rates are finally on the rise.

<https://www.cio.com>

- ◆ “The PMI research also identifies a number of factors common to the Champion organizations, says Langley. Besides a focus on **benefits realization**, champion organizations are more likely to focus on **project management talent**, have **at least one PMO** [project management office] within their organization, drive **executive sponsorship** and use **agile approaches to project management**, the research revealed.”

Software Engineering Process

A *software engineering process model* is a set of (1) *activities*, (2) *techniques*, (3) *deliverables* and (4) *tools* so that software engineers and project managers follow the (5) *best practices* (e.g., pair programming, agile methods) to use the items (1)–(4) to manage software systems

SE Process — Activities

- ◆ Collecting user requirements
- ◆ Designing software
- ◆ Coding/Implementation
- ◆ Testing
- ◆ Deploying the software at user sites
- ◆ Maintain software
- ◆ Configuration Management
- ◆ Project Management

Exemplified Software Requirements

User Requirements

Set of Natural Language Requirement:

Requirements - Train Station System

Develop a system to model the behavior of a Train-Station. You need to model a train entering the station from the north and then leaving the station to the south. A crossing with boom gates and flashing red lights is located just south of the station. There is a signal to the north of the station that only allows a train to enter when the station is not occupied, that is, when the north signal is green. There is also an exit signal light that ensures the train can only leave the station when the boom gates are down. There is also a north detector that can detect the train approaching the station region from the north. And, there is an exit detector that detects when a train leaves to the south. The following requirements capture the desired behavior.

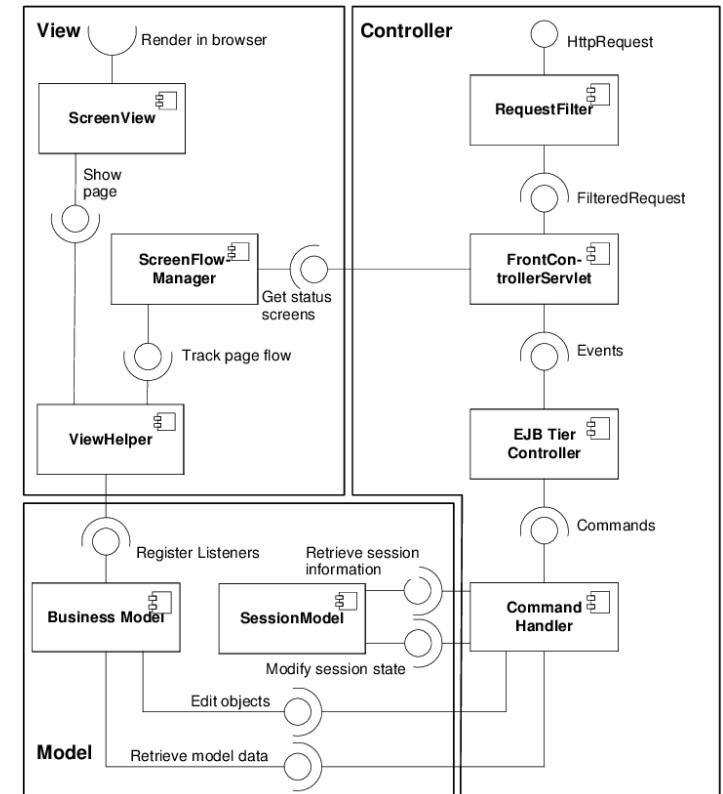
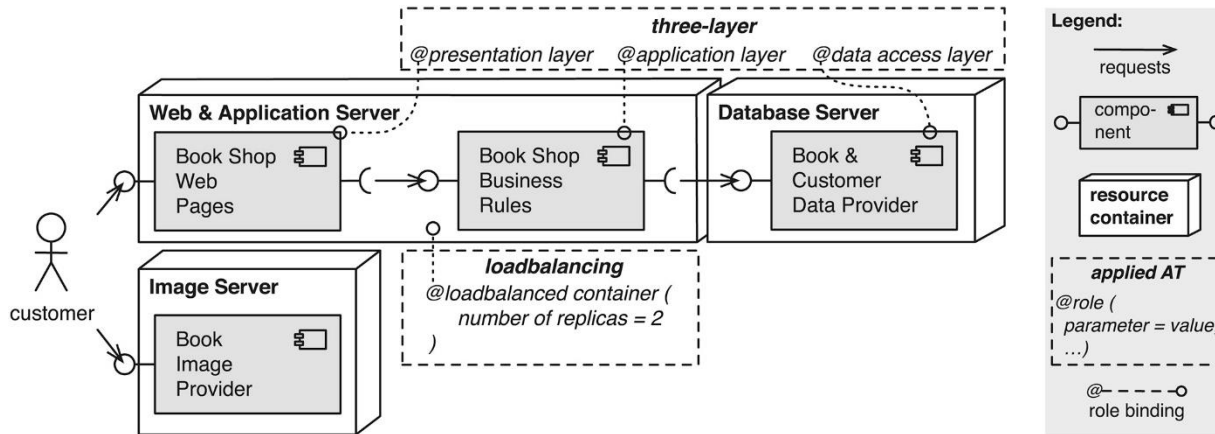
- R1.** Initially the station is not occupied. The north signal turns green whenever the station is not occupied. Whenever the north signal is green a train may approach from the north. When approaching from the north a train is detected, by the north detector, which causes the north signal to turn red.
- R2.** When the north detector detects a train it causes the crossing lights to start flashing red. At the same time, a timer starts timing and when it times out it causes the boom gates to be lowered after which the exit light turns green.
- R3.** After the train is detected the north detector, it subsequently arrives at the station, the doors open, the people disembark, and then the doors close.
- R4.** After the doors close the train may leave the station only when and if the exit light is green. When the train leaves the station, heading south, it is detected by the exit detector which means the station is again not occupied. This causes the north signal to turn green and the exit light to turn red. When the exit detector detects the train leaving, it also causes the boom gates to be raised and then the crossing lights to stop flashing red.

For the purposes of the exercise ignore trains approaching the station from the south. This additional requirement can be integrated later as a separate exercise. Also ignore situations where the train does not stop at the station - this too requires some refinements to the requirements.

Non-functional Requirements

- Product requirement
 - 4.C.8 It shall be possible for all necessary communication between the APSE and the user to be expressed in the standard Ada character set
- Organisational requirement
 - 9.3.2 The system development process and deliverable documents shall conform to the process and deliverables defined in XYZCo-SP-STAN-95
- External requirement
 - 7.6.5 The system shall not disclose any personal information about customers apart from their name and reference number to the operators of the system

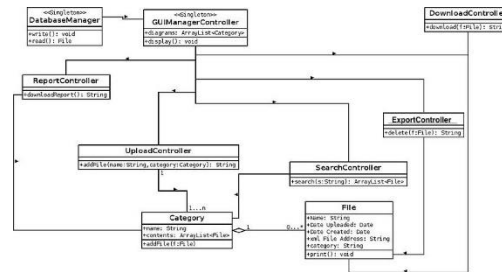
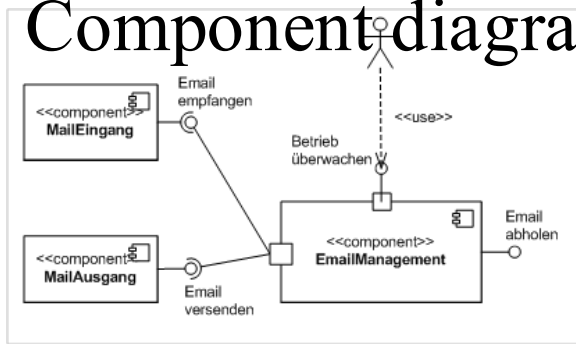
Exemplified Software Architecture



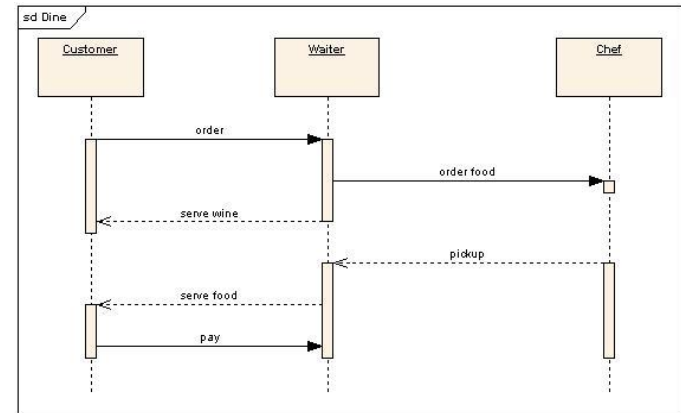
Exemplified Software Designs

Static view:

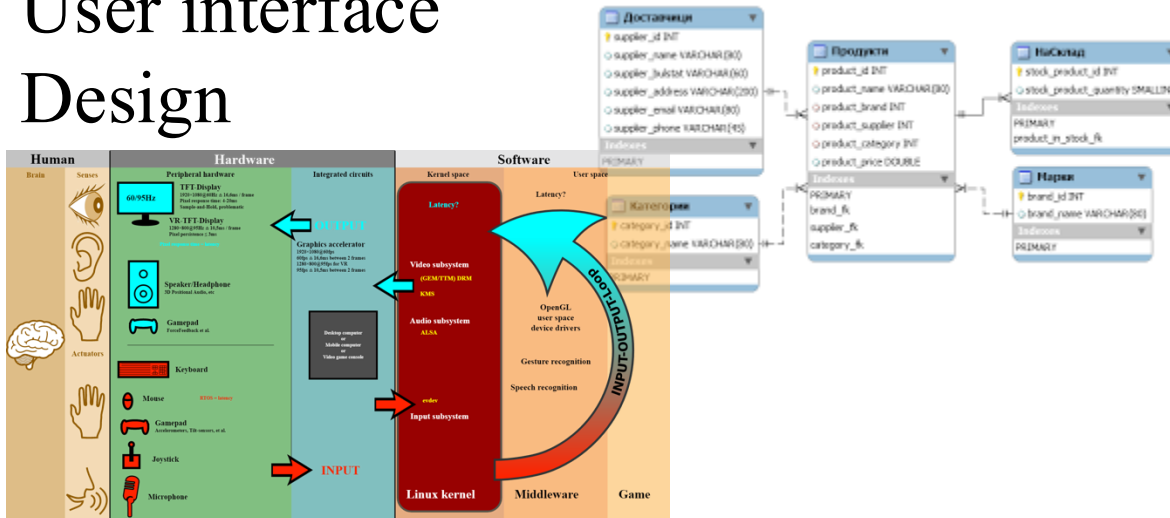
Component diagram Class diagram



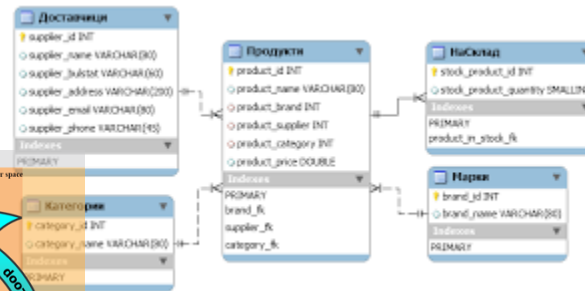
Dynamic view:
sequence diagram



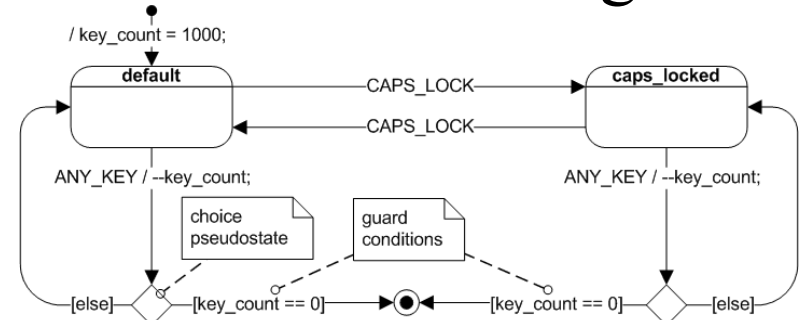
User interface
Design



Database scheme

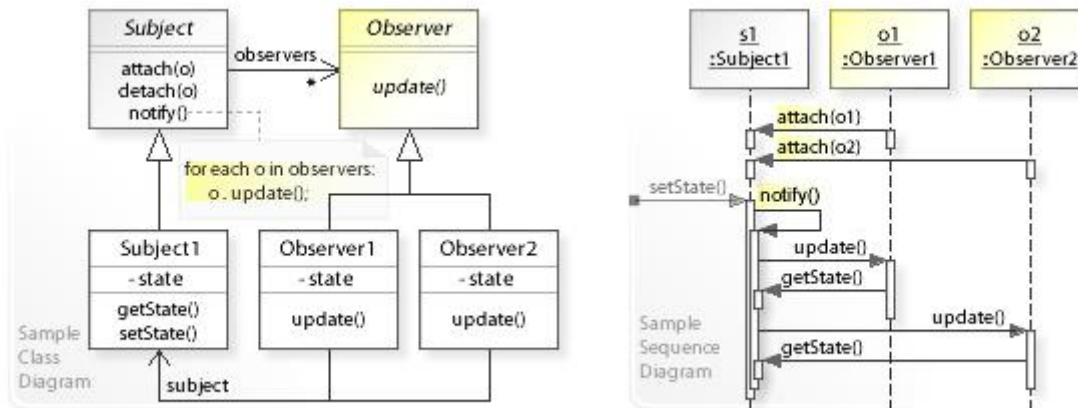


State transition diagram

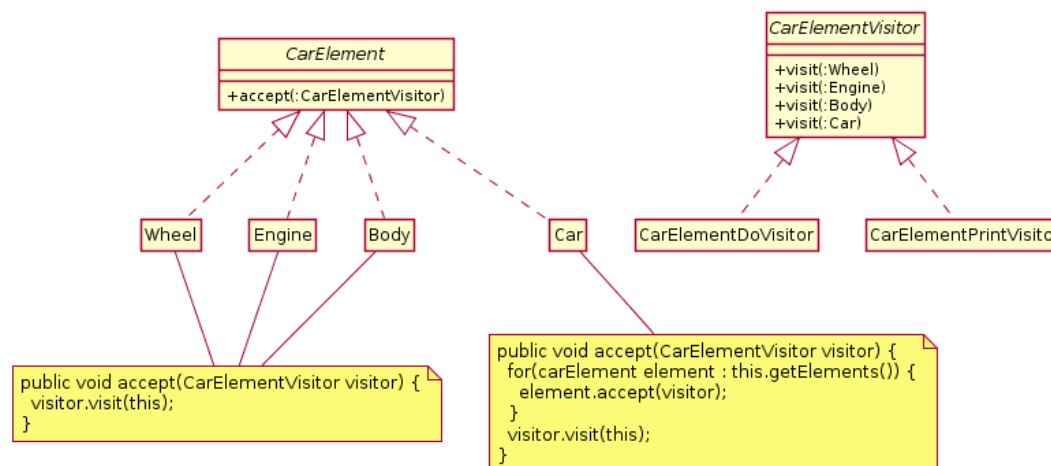


Exemplified Software Design Patterns

observer pattern



visitor pattern



Exemplified Coding

```
int risky_programming(char *input) {  
    char str[1000];  
  
    // ...  
  
    strcpy(str, input); // Copy input.  
  
    // ...  
}
```

Bad coding

```
int secure_programming(char *input) {  
    char str[1000+1]; // One more for the null character.  
  
    // ...  
  
    // Copy input without exceeding the length of the destination.  
    strncpy(str, input, sizeof(str));  
  
    // If strlen(input) >= sizeof(str) then strncpy won't null terminate.  
    // We counter this by always setting the last character in the buffer to NUL,  
    // effectively cropping the string to the maximum length we can handle.  
    // One can also decide to explicitly abort the program if strlen(input) is  
    // too long.  
    str[sizeof(str) - 1] = '\0';  
  
    // ...  
}
```



Exemplified Testing

```
import static org.junit.jupiter.api.Assertions.assertEquals;

import example.util.Calculator;

import org.junit.jupiter.api.Test;

class MyFirstJUnitJupiterTests {

    private final Calculator calculator = new Calculator();

    @Test
    void addition() {
        assertEquals(2, calculator.add(1, 1));
    }

}
```

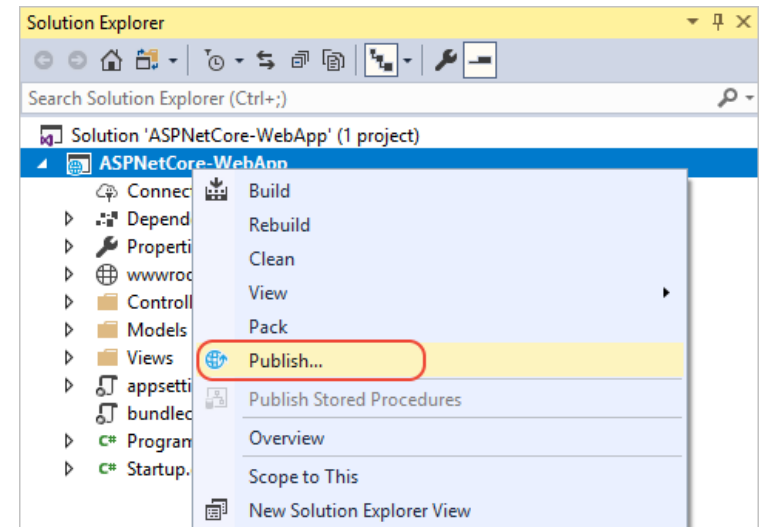
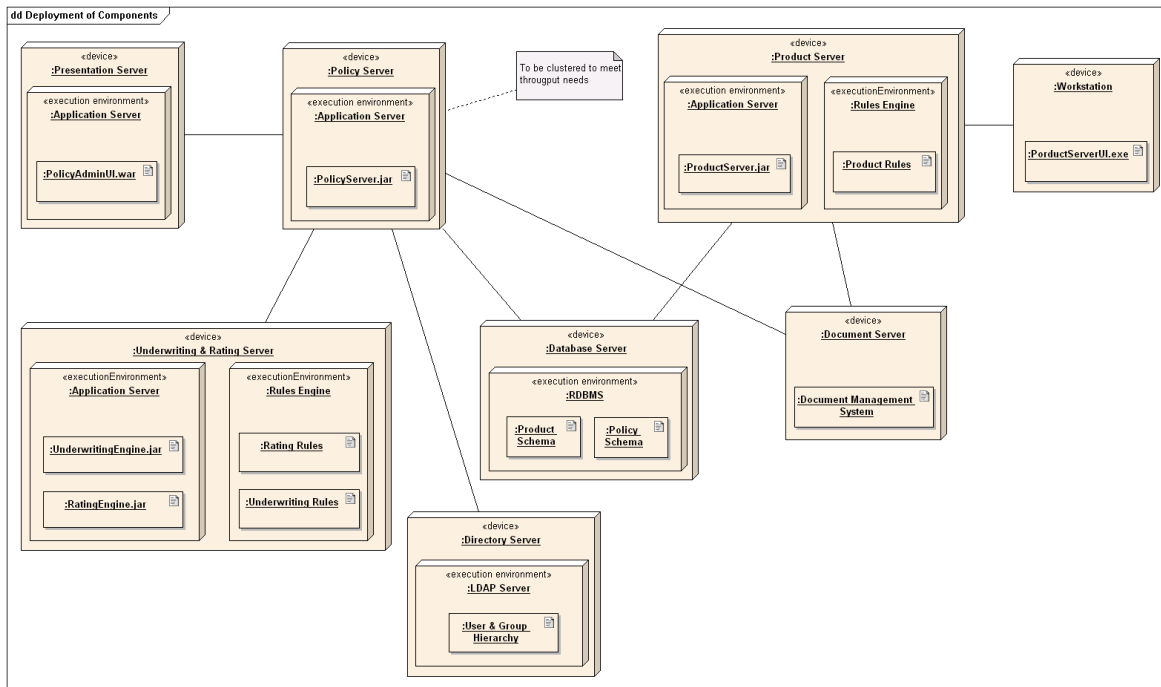
Unit Testing

Unit Testing: Test one class or just one function

Integration Testing: Test the interaction of multiple classes or functions

System Testing: Test from user input to system output in end- to-end manner

Exemplified Software Deployment



Deployment diagram: Map software component to run on hardware device (or hardware virtualization)

Write scripts or use a tool to publish it

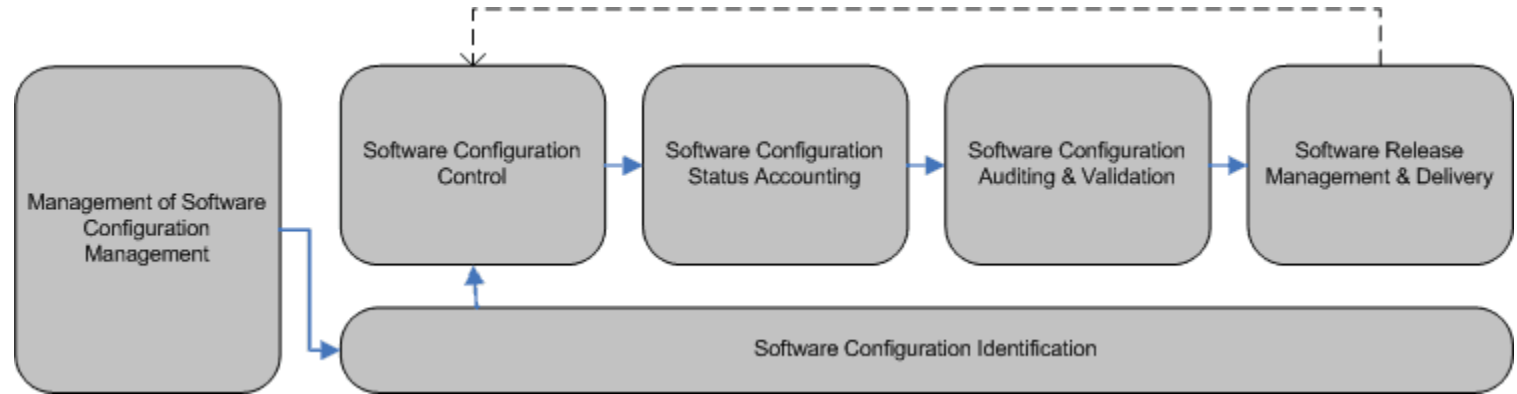
Exemplified Software Maintenance

The image displays two software interfaces side-by-side. On the left is the Jira 'Teams in Space' project board, showing a Kanban-style workflow with columns for 'TO DO 5', 'IN PROGRESS 5', 'CODE REVIEW 2', and 'DONE 8'. Each column contains task cards with details like 'Engage Jupiter Express for outer solar system travel' and 'Requesting available flights is now taking > 5 seconds'. On the right is a screenshot of the KDE Bug Tracking System in a Mozilla Firefox browser window. The browser address bar shows the URL: `http://bugs.kde.org/buglist.cgi?bug_status=NEW&bug_status=ASSIGNED&bug_status=REOPENED&bug_status=...`. The KDE Bug Tracking System page shows the date and time as 'Sat Sep 3 14:16:19 MEST 2005' and lists 64 bugs found. Below this is a table of bug details.

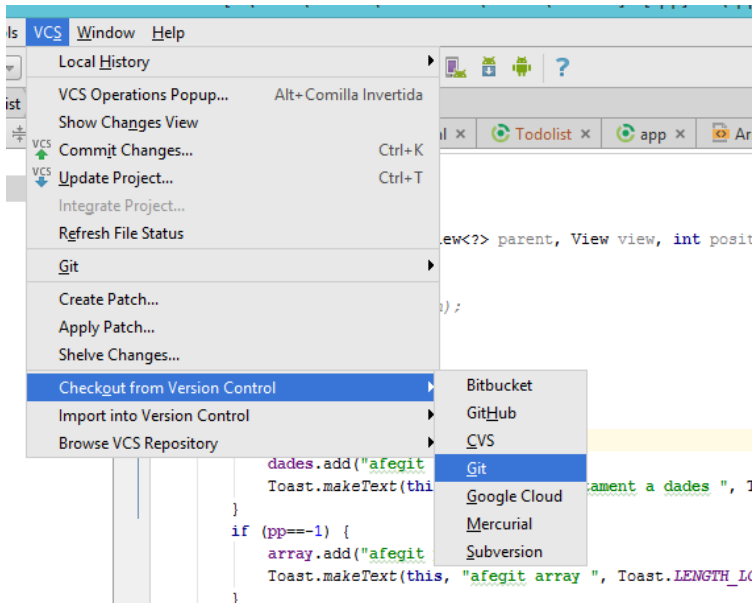
ID	Sev	Pri	Plt	Owner	State	Result	Summary
111909	wis	NOR	uns	kmail-devel kde org	NEW		printing multiple messages at once
111910	nor	NOR	Com	coolo kde org	NEW		Can't open links in whatsthis help
111911	wis	NOR	uns	kopete-devel kde org	NEW		support for drag and drop of attachment files in kopete
111912	nor	NOR	uns	linuxphreak gmx co uk	UNCO		wrong display of disk usage
111915	wis	NOR	SuS	konq-bugs kde org	UNCO		no crystalsvg key_enter.png available in 32x32
111916	nor	NOR	uns	andrew chant utoronto ca	UNCO		Number of enemy balls is always one no matter what it is ...
111917	nor	NOR	Gen	coolo kde org	UNCO		am_edit is tagging comment lines in Makefile.in
111918	nor	NOR	SuS	kdevelop-bugs kdevelop org	UNCO		setAttribute on form elements does not work
111919	nor	NOR	uns	kopete-devel kde org	UNCO		Contact protocol status icons shown offline with "plug"
111920	cra	NOR	SuS	digikam-devel kde org	UNCO		crash with signal 11 when sending album to trash
111922	wis	NOR	uns	konq-bugs kde org	NEW		file search dialog - default search period too long

Exemplified Configuration Management

◆ Process

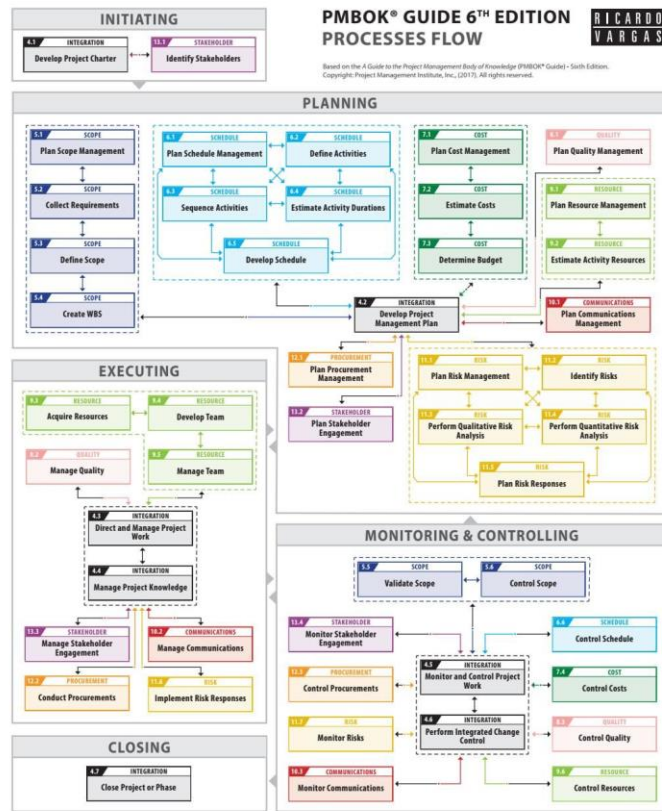


Code version control



Exemplified Project Management

PMBok



PMBOK® GUIDE 6TH EDITION - 49 PROJECT MANAGEMENT PROCESSES

Copyright: A Guide to the Project Management Body of Knowledge (PMBOK®) Guide - Sixth Edition, Project Management Institute, Inc. (2017). All rights reserved.
 Explanatory Note: This flow does not replace the need for reading the PMBOK® Guide. The PMBOK® Guide contains a deep exploration of all processes including inputs, tools & techniques and outputs that are not listed in this flow.
 Adaptation: Ricardo Vargas
 Graphic Design: Sérgio Alves Lima Jardim

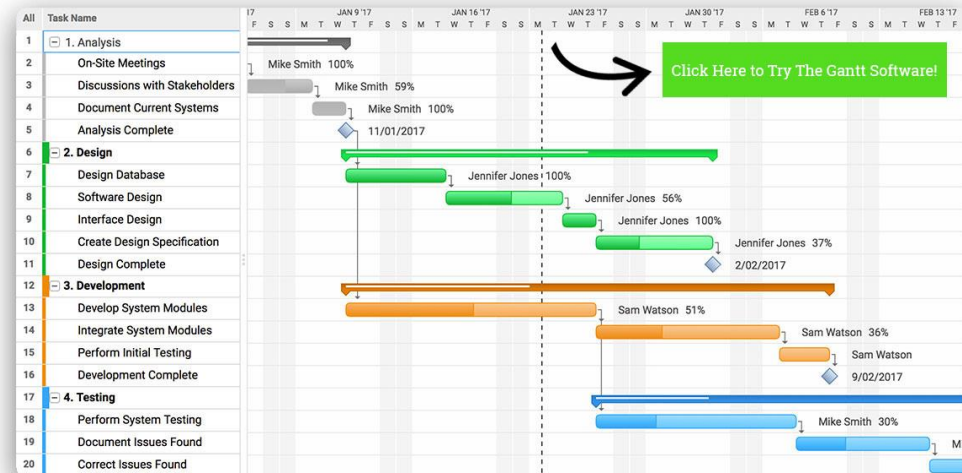
ricardo-vargas.com

Download this processes flow at <http://rvarg.as/pmbok6en>

Become a member of the Project Management Institute (PMI) and download the PMBOK® Guide and other standards at:

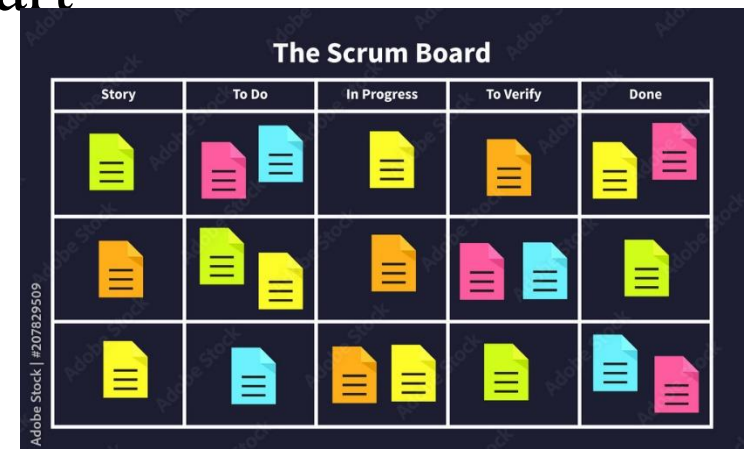
www.pmi.org

Courtesy <https://www.projectmanager.com/gantt-chart>



Gantt Chart

Agile-based task board



SE Process — techniques

		Example
◆ Collecting user requirements	➡	◆ Use cases / user stories / use case diagram / meeting
◆ Designing software	➡	◆ UML/ patterns/ principles/...
◆ Coding/Implementation	➡	◆ Java / C++ / ...
◆ Testing	➡	◆ Unit testing / debugger / ...
◆ Deploying the software at user sites	➡	◆ Standalone software/Software plug-in/app/web services/...
◆ Maintain software	➡	◆ Bug reporting/software repository
◆ Configuration Management	➡	◆ Version control / change management software
◆ Project Management	➡	◆ Work breakdown structure/ task scheduling algorithm

SE Process — deliverables

Example

◆ Use cases / user stories / use case diagram	⇒	◆ Requirement document
◆ UML/ patterns/ principles/...	⇒	◆ Software design document
◆ Java / C++/...	⇒	◆ Code listing and test script
◆ Unit testing / debugger /...	⇒	◆ Quality assurance (QA) report
◆ Software plug-in/applet architecture	⇒	◆ Software at user site
◆ Bug reporting/software repository	⇒	◆ Bug report and s/w release
◆ Version control / change management software	⇒	◆ Code change / patch / change history report
◆ Work breakdown structure/ task scheduling algorithm	⇒	◆ Project schedule and status tracking

SE Process — tools

Example

◆ Use cases / user stories / use case diagram	⇒	◆ UML tool / Requirement modeling tool / Word
◆ UML/ patterns/ principles/...	⇒	◆ UML tool / SonarQube
◆ Java / C++/...	⇒	◆ Visual Studio/Eclipse
◆ Unit testing / debugger /...	⇒	◆ JUnit/ debugger /fuzzer ...
◆ Software plug-in/app/web services	⇒	◆ Installer/plug-in framework
◆ Bug reporting/software repository	⇒	◆ Bug reporting system
◆ Version control / change management software	⇒	◆ Apache subversion / git
◆ Work breakdown structure/ task scheduling algorithm	⇒	◆ MS Project

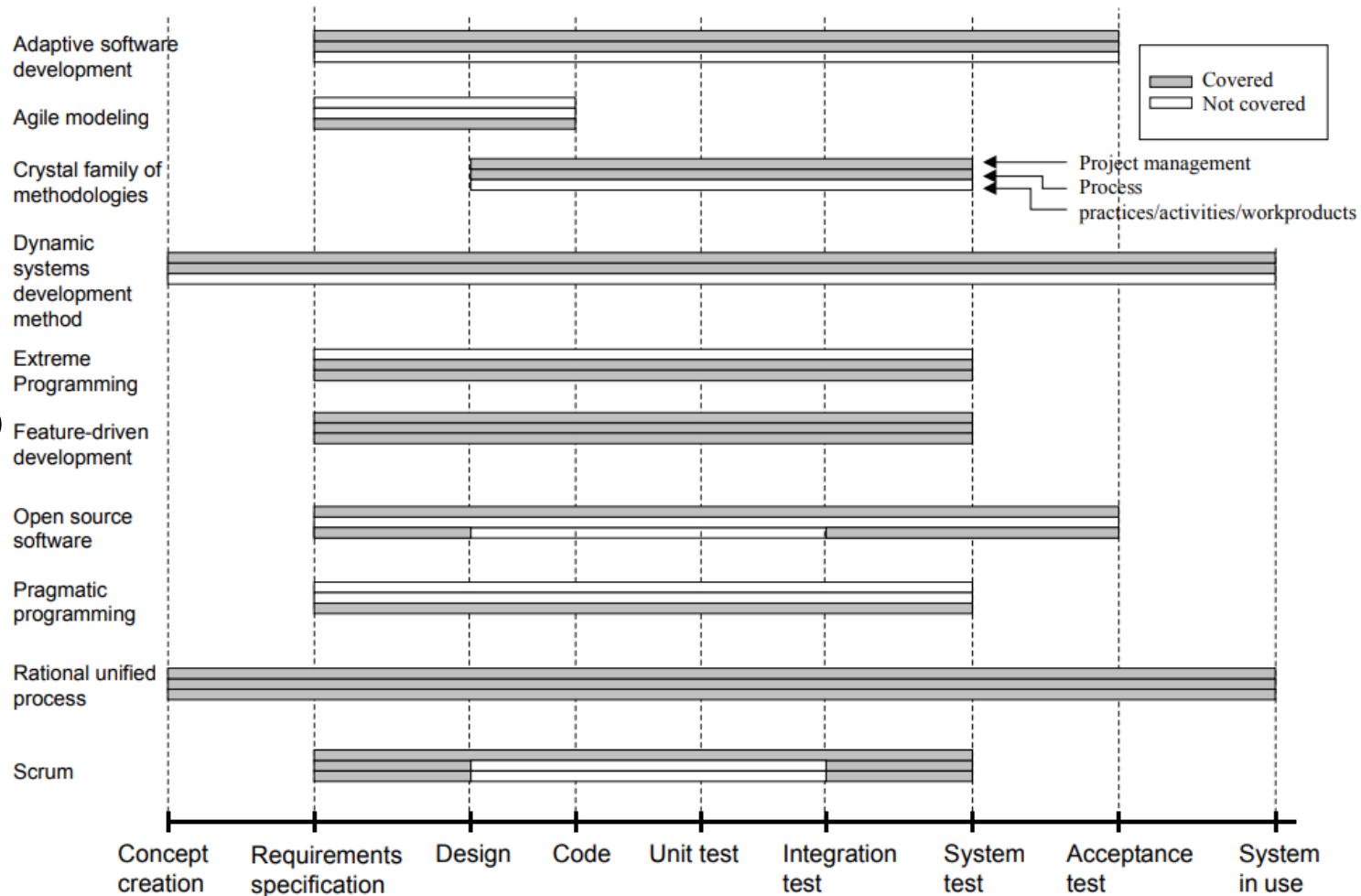
SE Process — Best Practices

- ◆ Different procedures to organize activities define different types of software processes
- ◆ E.g., [see the details in the next few slides]
 - Waterfall model
 - V-shape waterfall model
 - Incremental software development model
 - Prototyping
 - Spiral model
 - Unified Process (UP)
 - **Agile Methods**

1960s
1980s
2000s
↓ 2010s

Many Kinds of Agile Methods

All 10 are agile methods.

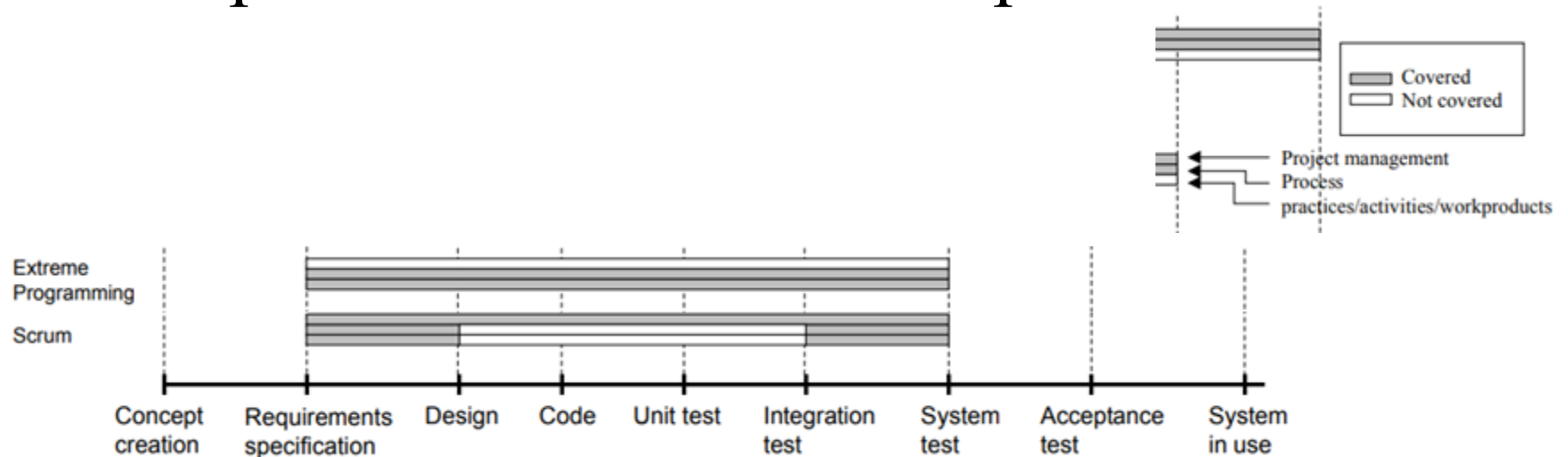


1-Minute Q&A

- ◆ **Question 1:** On last slide, which agile methods include the project management activities?
- ◆ If an agile method require fewer specific tools, activities, process steps, intermediate products, then the agile method is more lightweight.
- ◆ **Question 2:** Are they any agile methods both lightweight and including project management activities? Name them

Extreme Programming vs Scrum

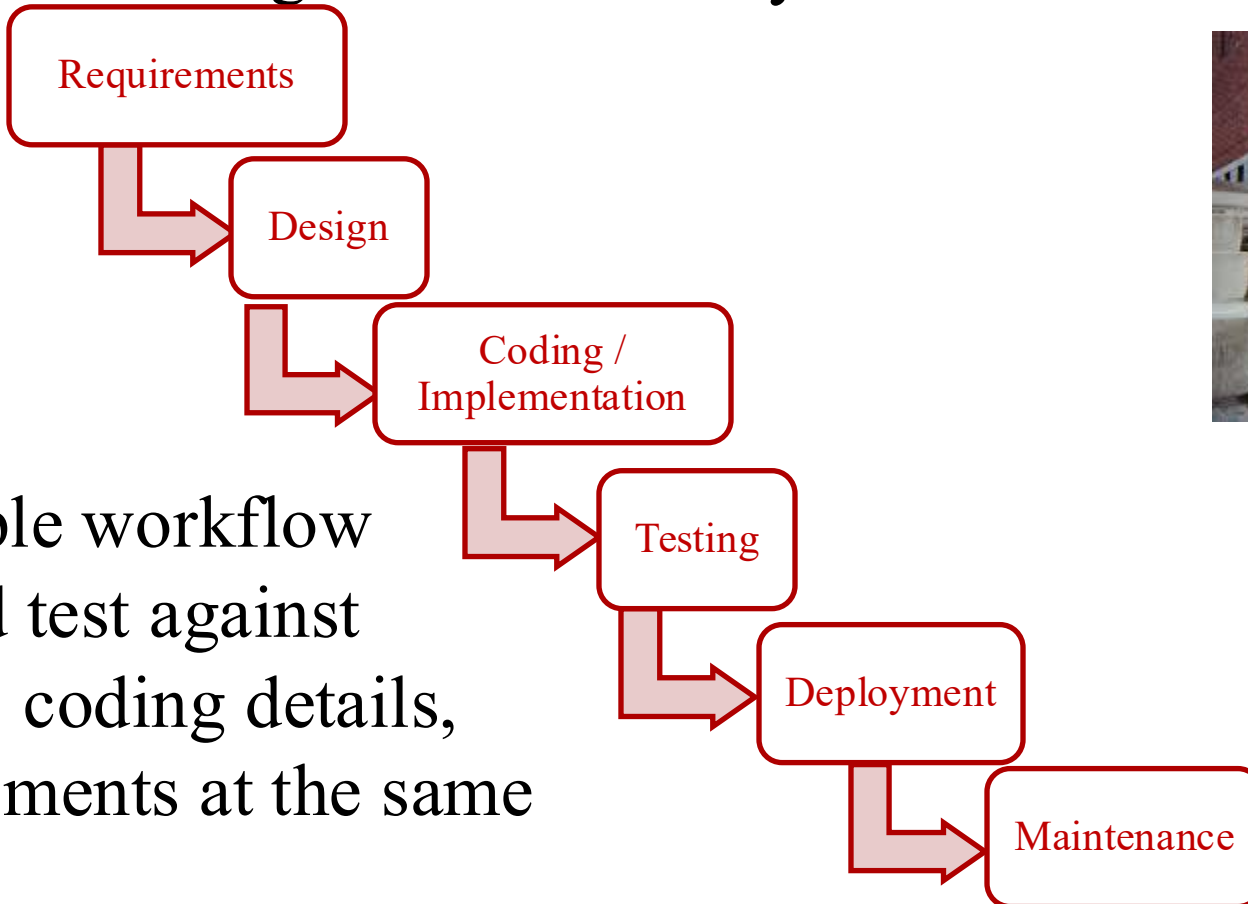
- ◆ XP has no PM.
- ◆ Scrum has PM (but a less tedious process than XP) and other practices/activities/work products



Brief Look on Process Models

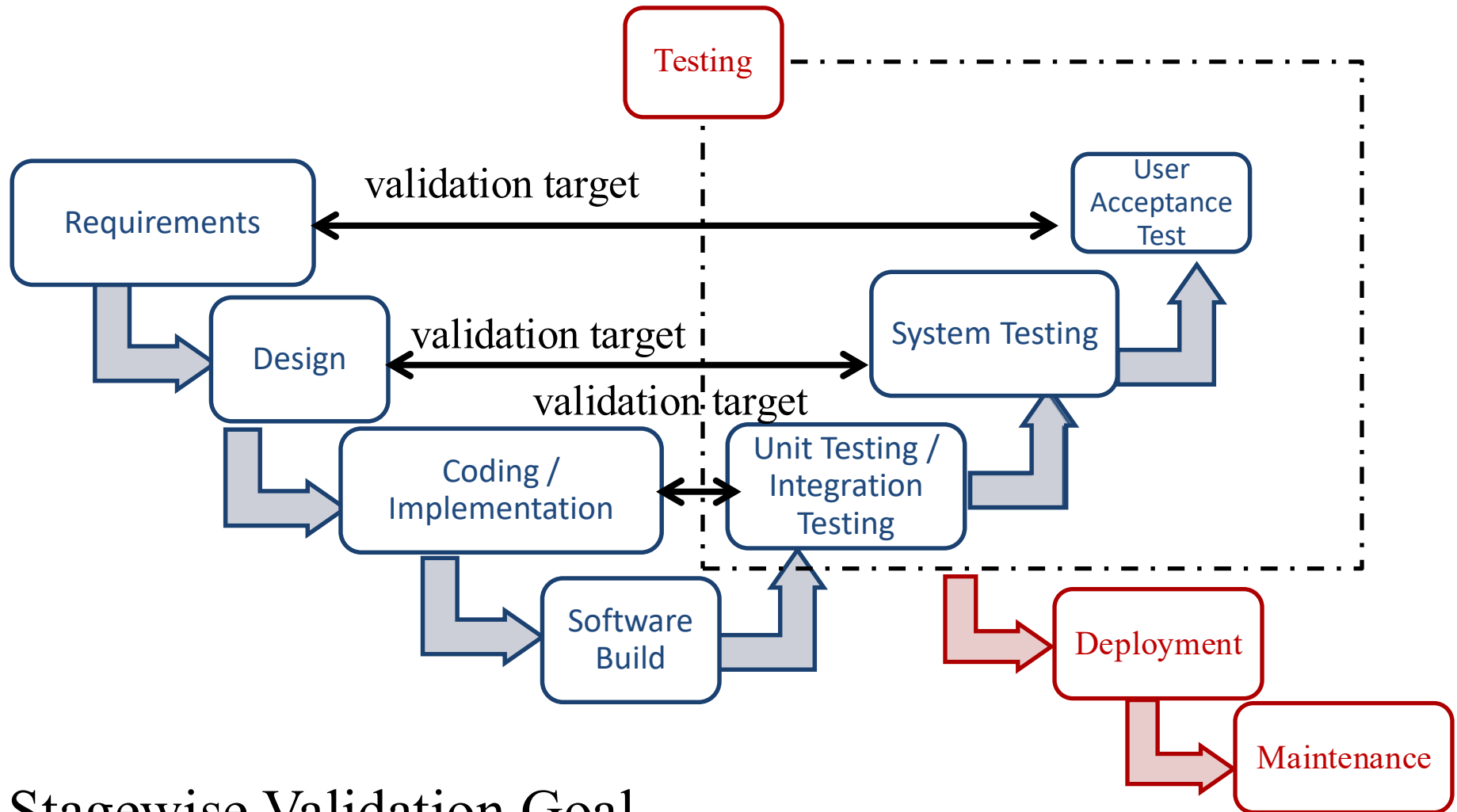
Waterfall Model

- ◆ *Completely* produce the full set of *deliverables* of each activity before starting the next activity.



+ Simple workflow
– Need test against design, coding details, requirements at the same time.

V-Shape Waterfall Model



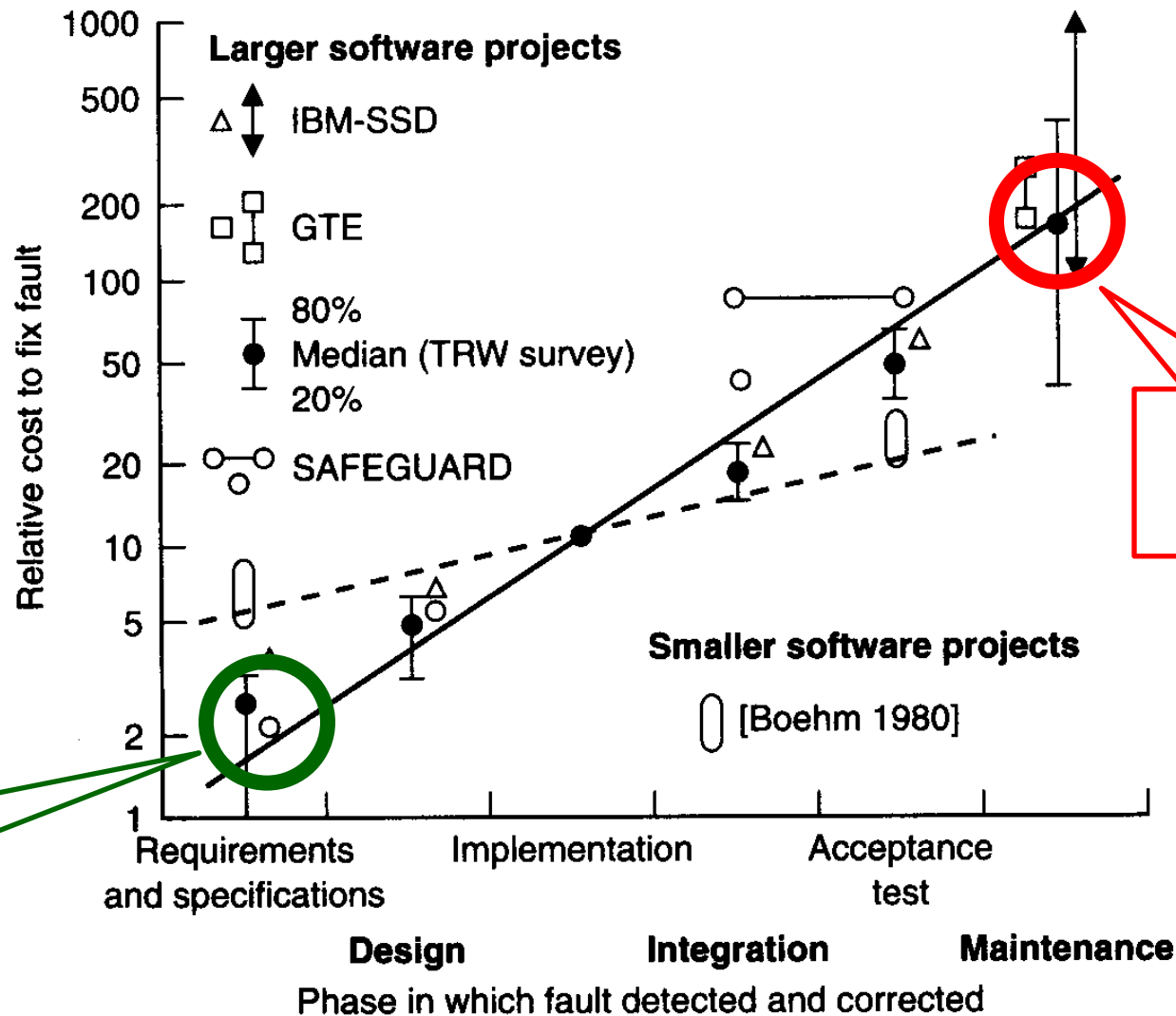
+ Stagewise Validation Goal

Problems of “Waterfall Model” and “V-Shape Waterfall Model”

1. False sense of clear-cut phases of activities.
 - E.g., Can we completely work out the actual requirement before we design our software?
 - It is unrealistic for real-world problems.
 - It is also very costly to correct the problems made in upstream activities. [see the next slide]
2. Nothing is done until they are all done
 - E.g., If we end the project at the design phase, then there is no any code delivered to the project clients, but our clients want to have something executable to help their own business!

Lesson Learnt:

Cost of Fixing Bugs at Earlier Stage is Cheaper



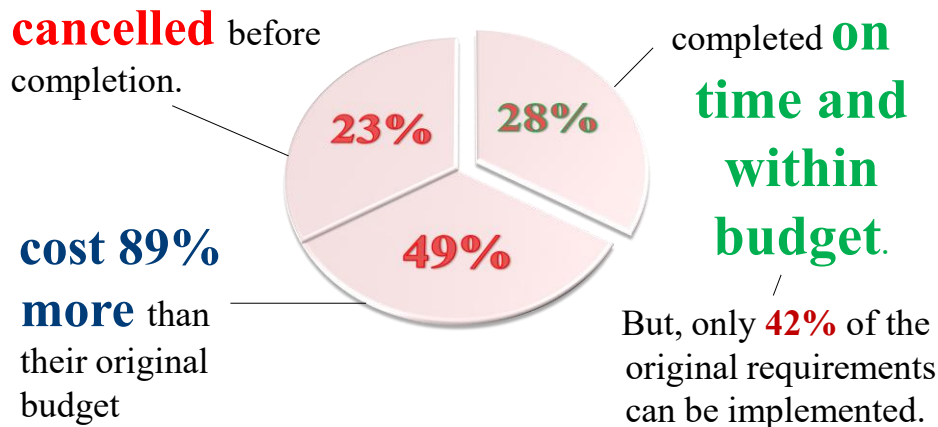
An artifact that can be validated should be validated as early as possible?

Implication on following the Waterfall Model

We would rather have a *gracefully degraded* subsystem

- a partial system, a system of lower quality, instead of no system at the end!

Facts: Software Development Project



An artifact of an early release subsystem that can be validated should be validated as early as possible? How?

Software Process Improvement

Current Trends (1)

- ◆ Some stages (activities) can be done *in parallel*
- ◆ *Example:* Run technical development and user training in parallel

- Development and validation within each parallel track.
- Each parallel track fits within a self-contained subsystem.
- Quicker for each parallel track to move forward, thereby earlier to rectify bugs => lower cost

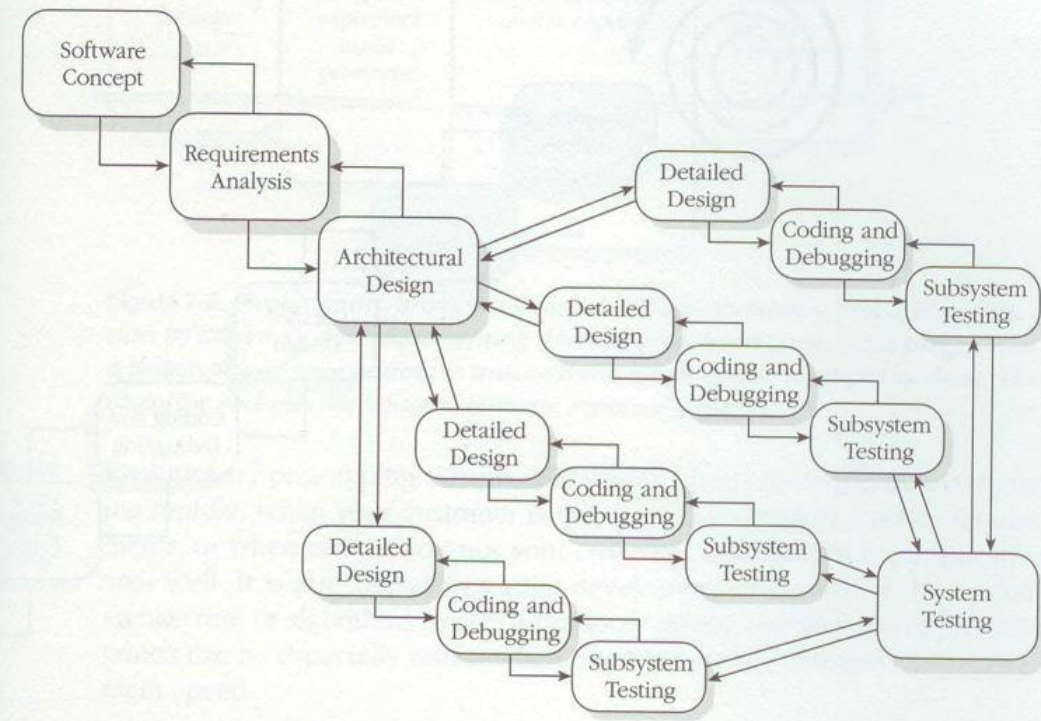


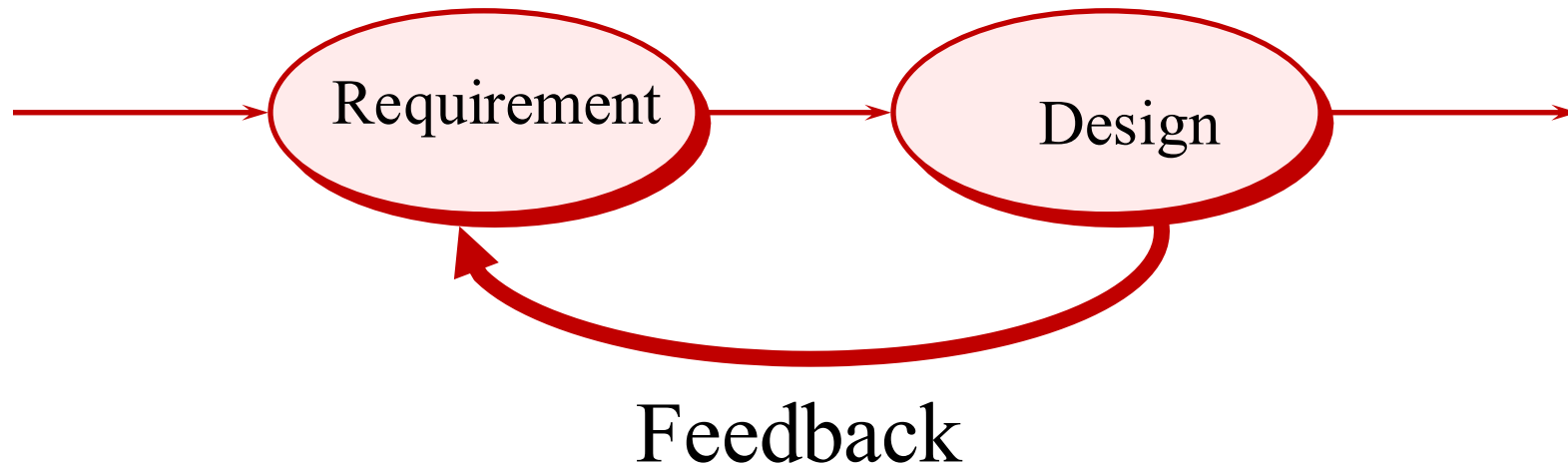
Figure 7-6. The waterfall model with subprojects. Careful planning can allow you to perform some of the waterfall's tasks in parallel.

Software Process Improvement

Current Trends (2)

◆ *Backward Iterations*

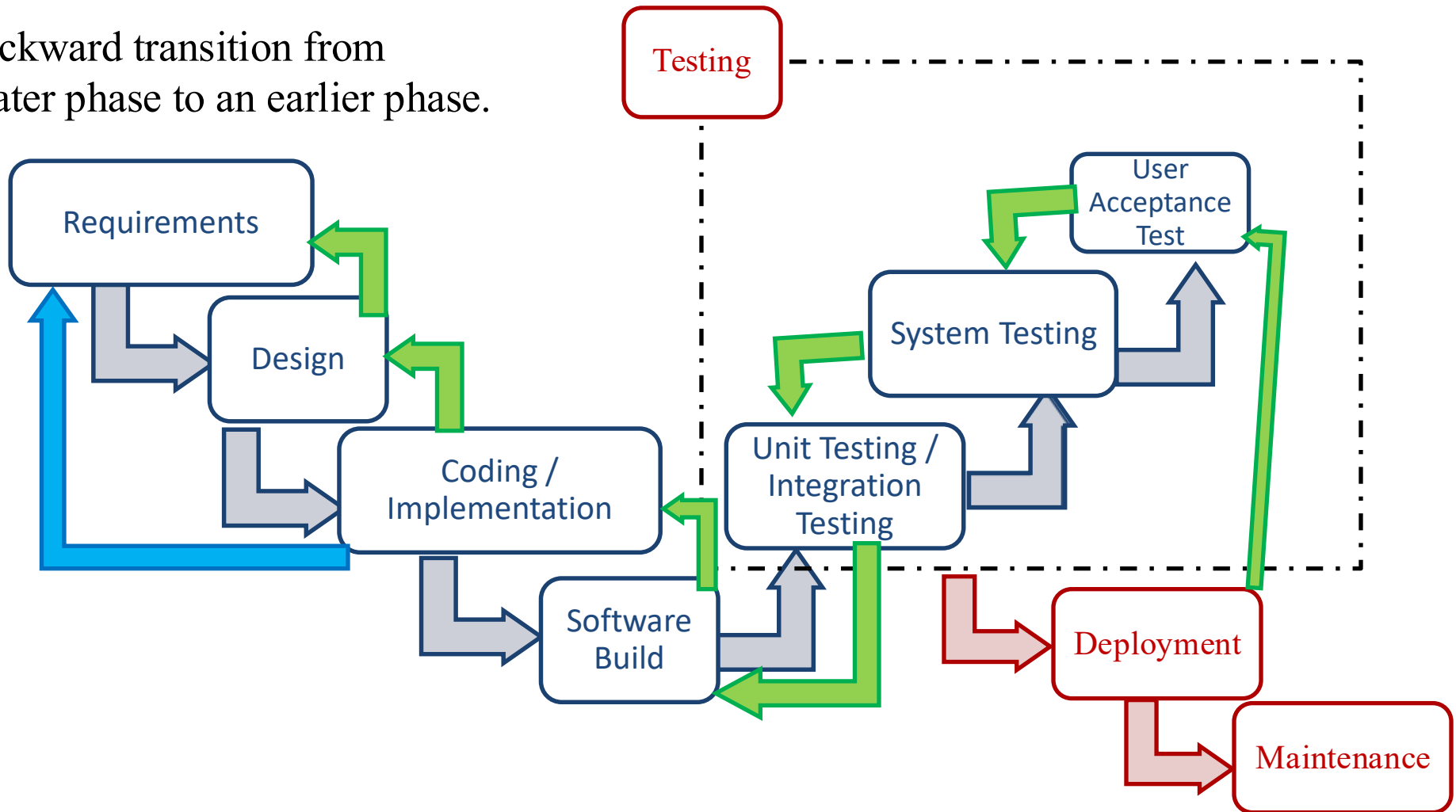
Example:



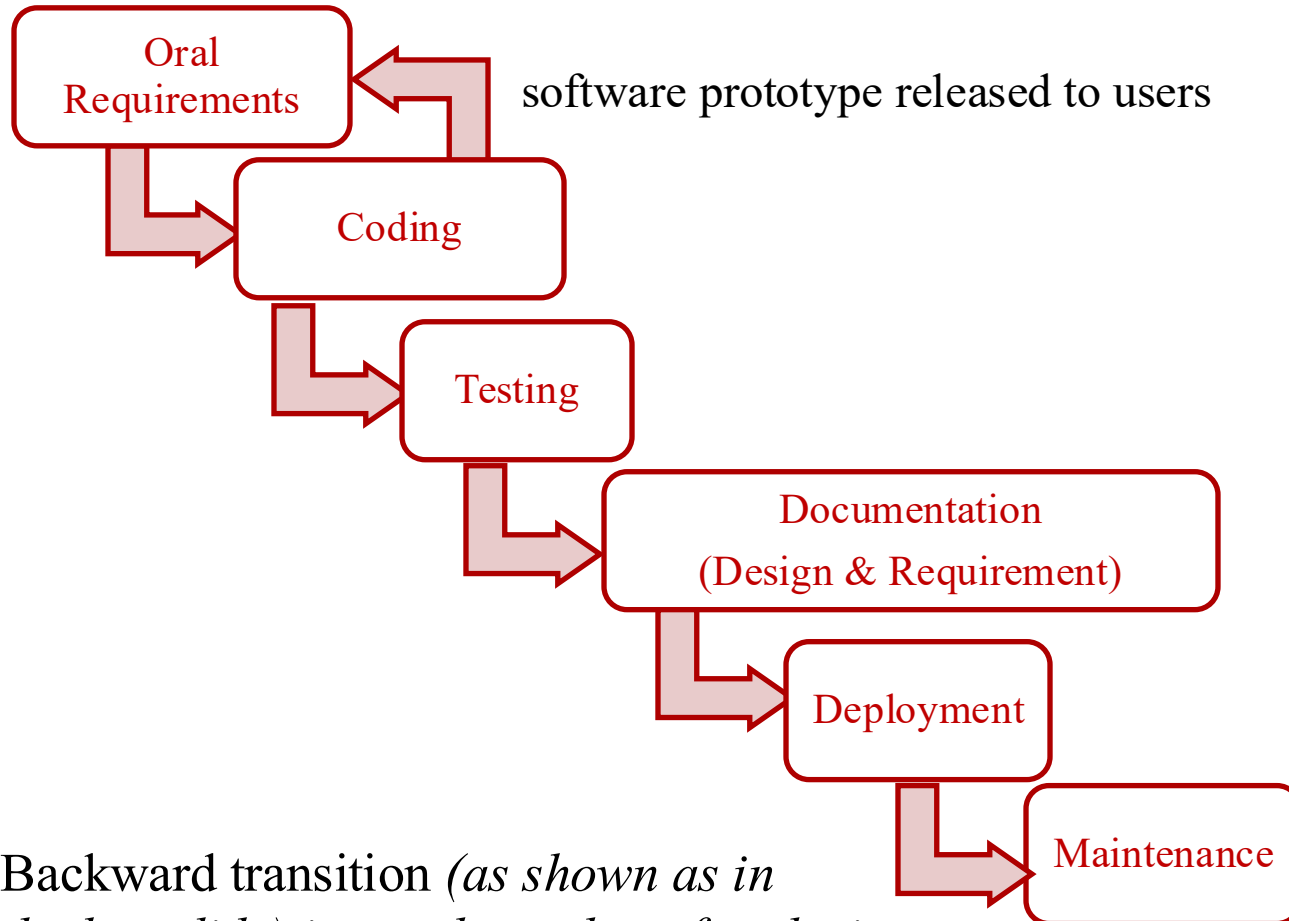
- Notices the direction of arrows.
- Improve earlier stages whenever necessary
- Avoid unnecessary rework or design bugs by working out on the more accurate requirements

V-Shape Waterfall Model

Backward transition from
a later phase to an earlier phase.



Prototyping



Backward transition (as shown as in the last slide) is not shown here for clarity.

Prototyping:

- + Code before design and requirement documentation

A prototype is a version of the final product that is likely buggy and only implements some features or merely the screen effects for users to “see” the product so that users can point out their requirements easier than expressing them in texts and UML.

Incrementally obtain and validate the user/system requirements before a proper software design stage.

Incremental Development

- ◆ Rather than using a parallel track to work on a smaller subsystem, another way is to deliver a smaller set of requirements and gradually enlarge the requirement set.
- ◆ Divide the set of requirements into subsets.

Implementing one subset as a program version v_1

then

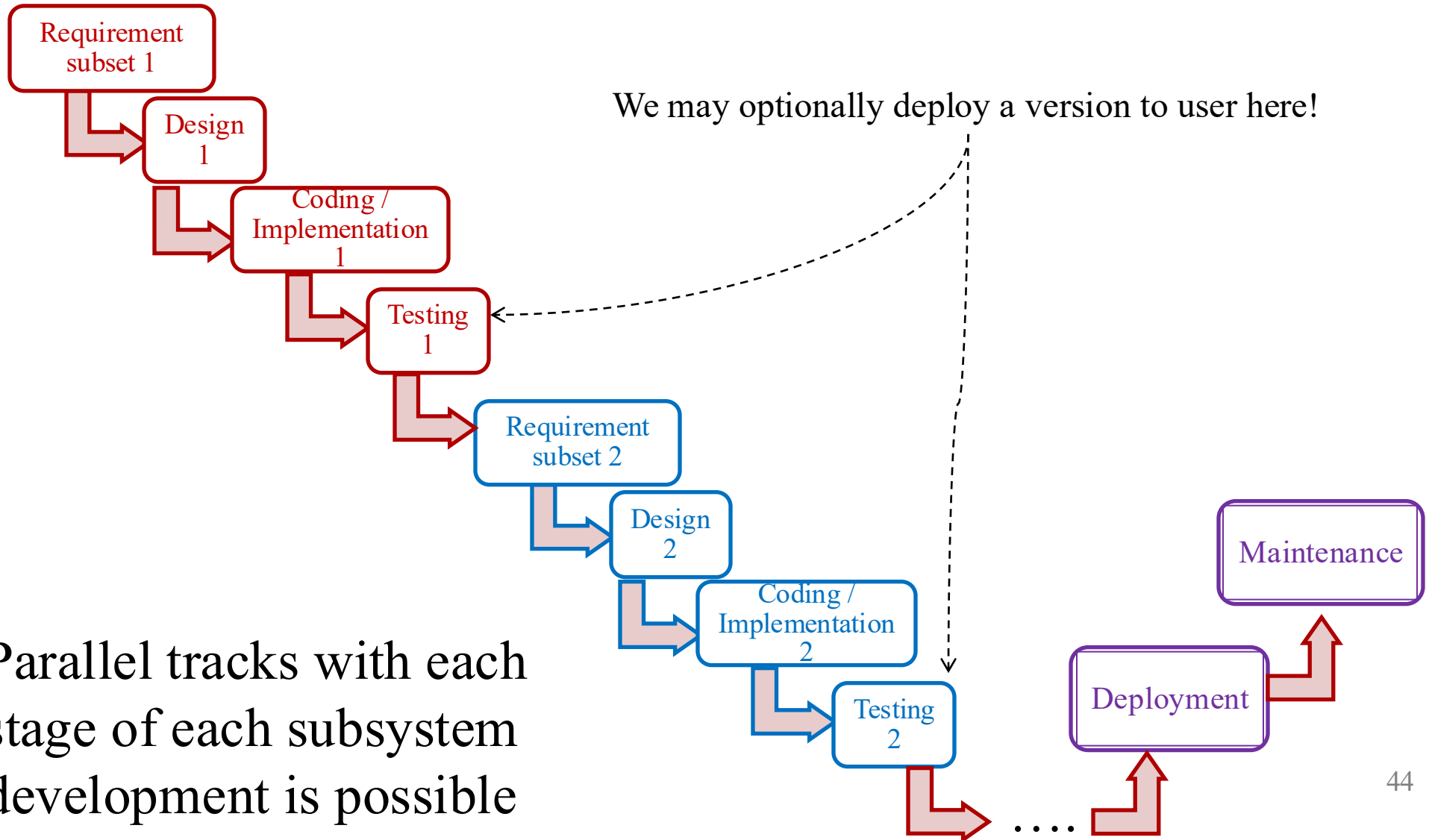
Implementing another subset on top of v_1 as version v_2

then

Implementing yet another subset on top of v_2 as version v_3

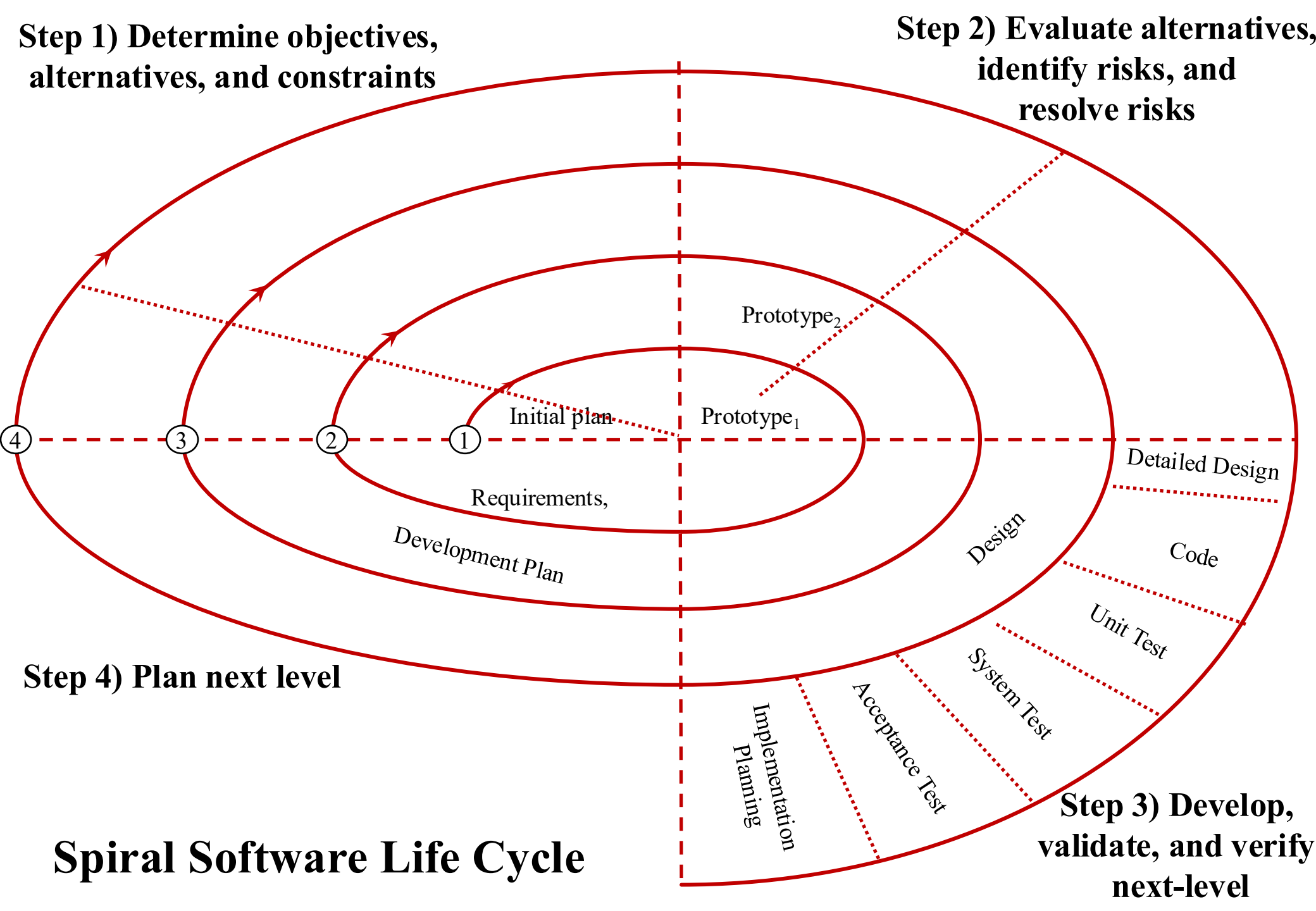
then ...

Incremental Development



Spiral Model

- ◆ Contains characteristics of the waterfall, prototype, and incremental development, but is more systematic.
- ◆ *Basic idea*: plan, do prototyping, revise the previous plans until we know the item *X* well through a series of iterations.



Software Process Improvement

Current Trends

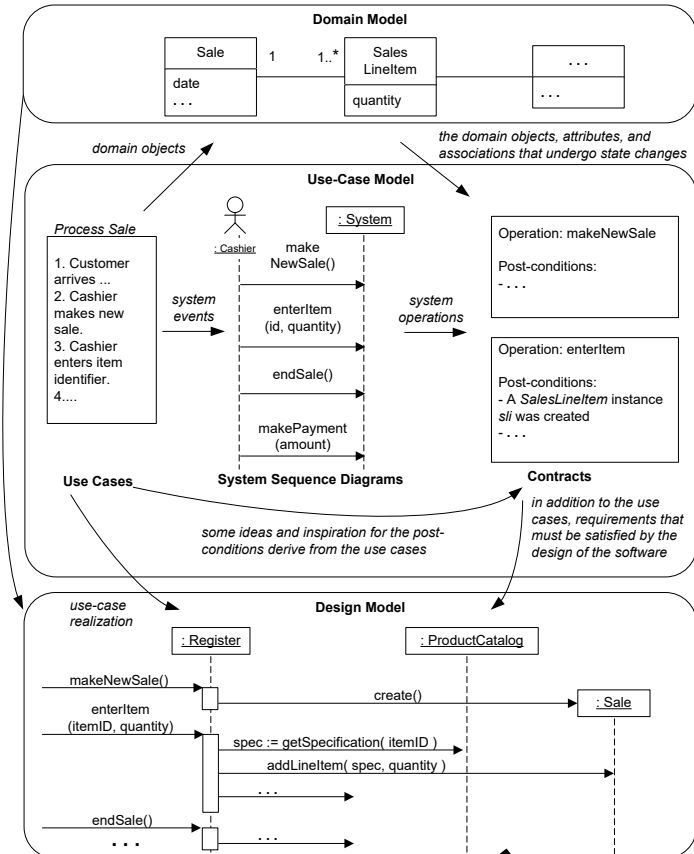
- ◆ *Model-Based Software Engineering Process*
(various modeling skills as the techniques)



Developer

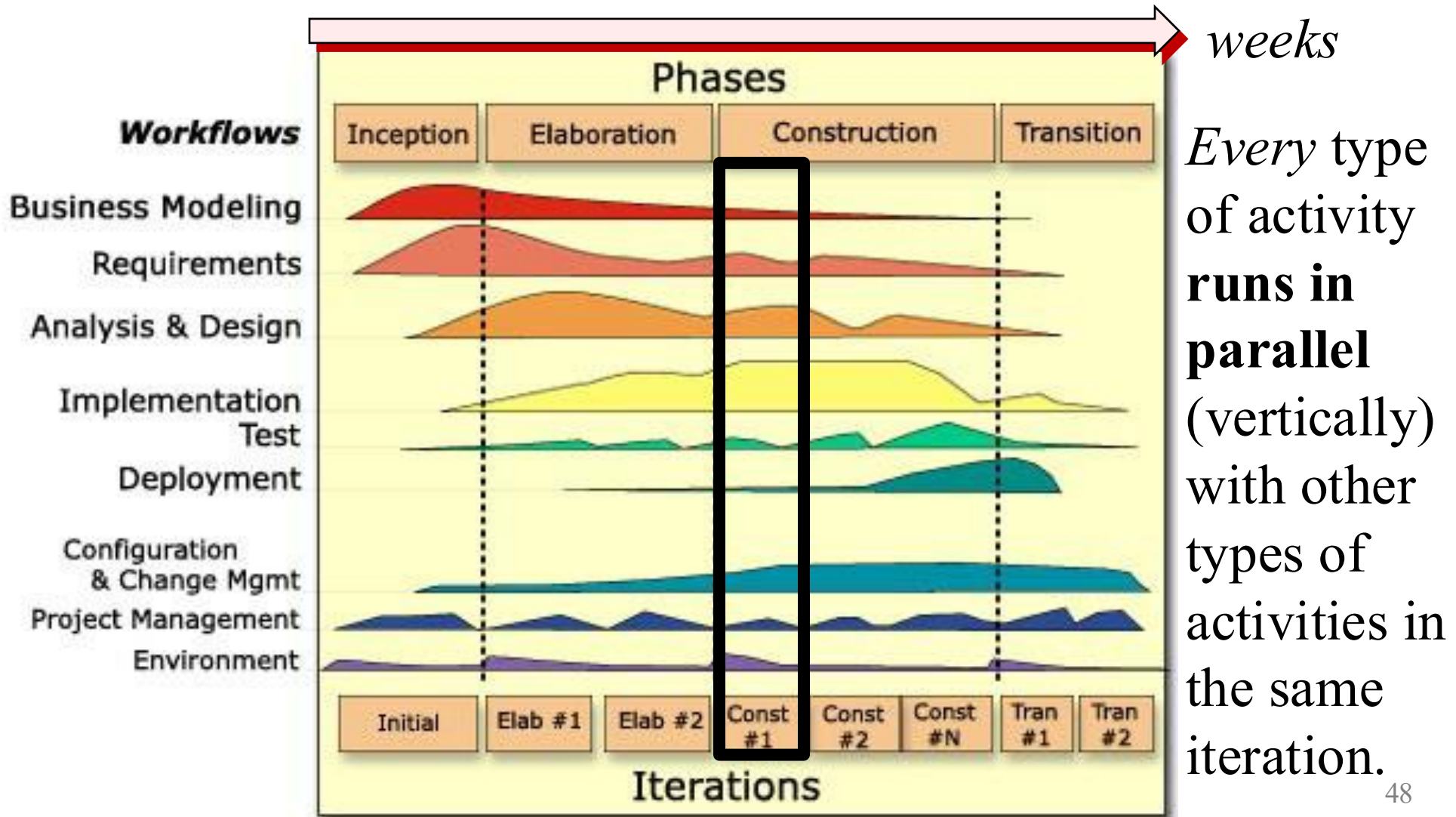
Interact

Interact



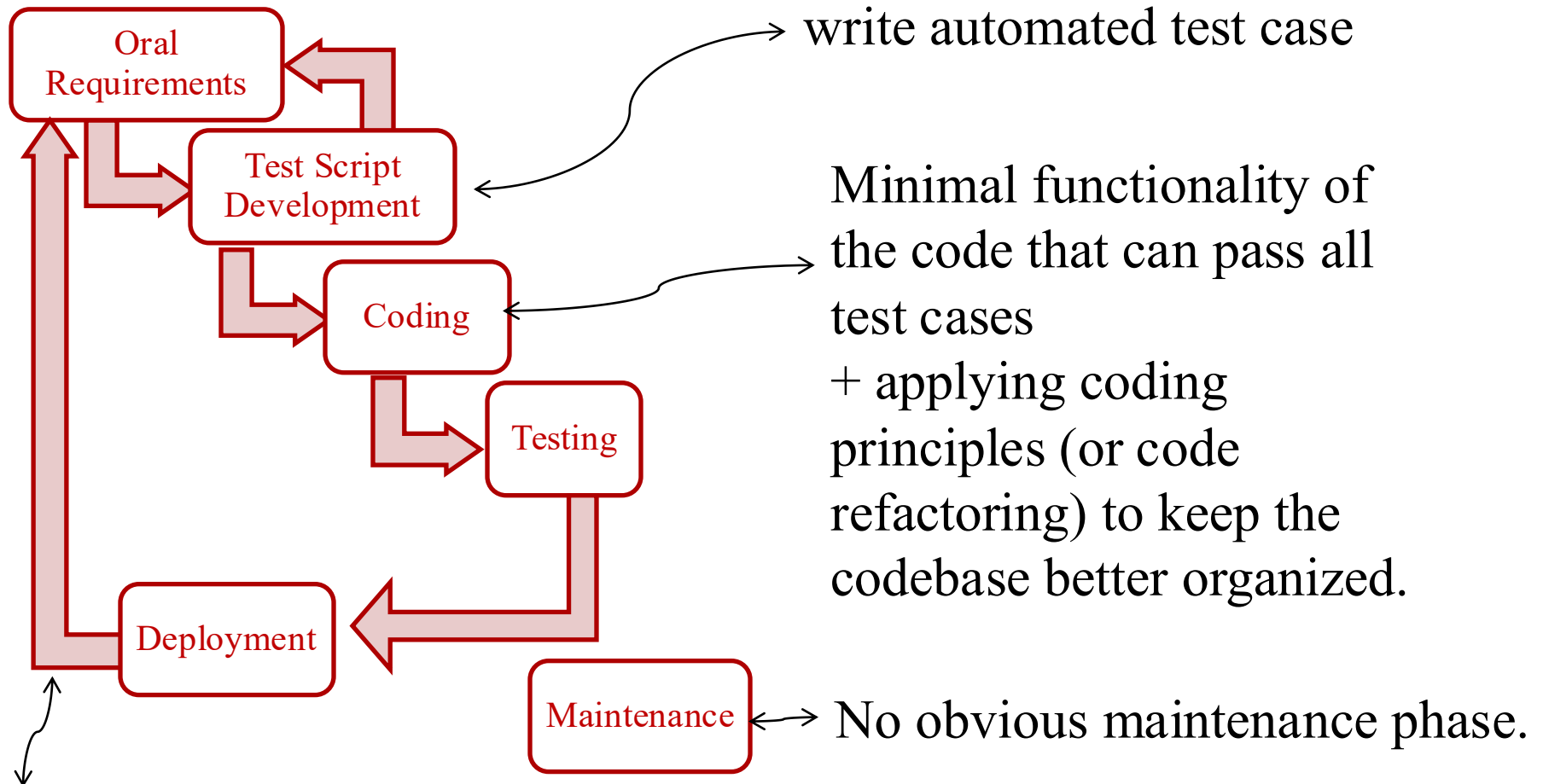
Construct

(Rational) Unified Process



All diagrams are rough sketches to ease communications.

Ideas of Test-Driven Development



Each iteration must be short in duration in terms of time.

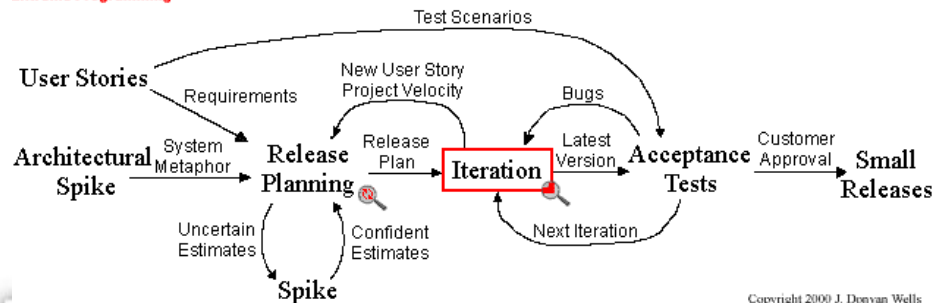
XP

- ◆ Extreme programming (XP) has five phases:
 - Planning
 - Meet with users, define user stories, and estimate story points. Plan for release through N iterations (for user stories delivery)
 - Design
 - Use Simple and consistent design sketch
 - Coding
 - Apply XP practice (see the next two slides)
 - Testing
 - Conduct automated unit tests and acceptance tests per user story
 - Feedback
 - PM and users determine the values of the user stories delivered by the implementation

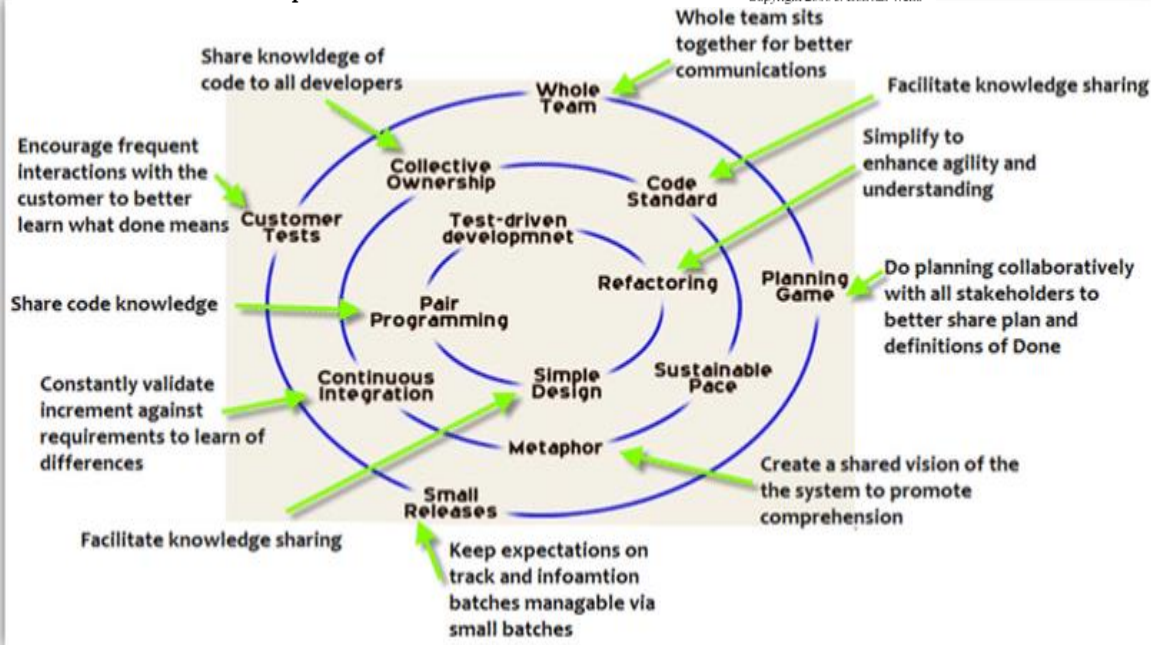
Extreme Programming XP [an Agile Method]



Extreme Programming Project



Copyright 2000 J. Donovan Wells



◆ Emphases:

- Generate feedbacks
- Embrace changes
- keep customers engaged
- short iteration
- Fix bugs early

XP practices



The Values of Extreme Programming

Feedback: We will take every iteration seriously by delivering working software early and often then listen carefully and make any changes needed. We will talk about the project and adapt our process to it, not the other way around.

Respect: Everyone gives and feels the respect they deserve as a valued team member.

Simplicity: We will do what is needed and asked for, but no more. This will maximize the value created for the investment made to date. We will take small simple steps to our goal and mitigate failures as they happen. We will create something we are proud of and maintain it long term for reasonable costs.

Communication: Everyone is part of the team and we communicate face to face daily. We will work together on everything from requirements to code. We will create the best solution to our problem that we can together.

What lessons have we learned about implementing XP so far?

<http://www.extremeprogramming.org/rules.html>



The Rules of Extreme Programming

Planning

- User stories are written.
- Release planning creates the release schedule.
- Make frequent small releases.
- The project is divided into iterations.
- Iteration planning starts each iteration.

Managing

- Give the team a dedicated open work space.
- Set a sustainable pace.
- A stand up meeting starts each day.
- The Project Velocity is measured.
- Move people around.
- Fix XP when it breaks.

Designing

- Simplicity.
- Choose a system metaphor.
- Use CRC cards for design sessions.
- Create spike solutions to reduce risk.
- No functionality is added early.
- Refactor whenever and wherever possible.



Coding

- The customer is always available.
- Code must be written to agreed standards.
- Code the unit test first.
- All production code is pair programmed.
- Only one pair integrates code at a time.
- Integrate often.
- Set up a dedicated integration computer.
- Use collective ownership.

Testing

- All code must have unit tests.
- All code must pass all unit tests before it can be released.
- When a bug is found tests are created.
- Acceptance tests are run often and the score is published.

XP guidelines

- ◆ **Planning:** programmers estimate efforts needed for implementing user stories and customer decides the scope and timing of release
- ◆ **Small release:** monthly, or daily for small fixes
- ◆ **Metaphor:** A shared story guides all developments by describing how the system works
- ◆ **Simple design:** use simplest possible solution
- ◆ **Testing:** use tests are implemented before the code. Customer write the functional tests.
- ◆ **Refactoring:** do refactoring frequently
- ◆ **Pair programming:** two people write code at one computer.
- ◆ **Collective ownership:** anyone can change any part of the code at anytime
- ◆ **Continuous integration:** integrate the code to the project codebase as soon as it is ready.
- ◆ **On-side customer:** customers are available full-time
- ◆ **Coding standards:** apply them.
- ◆ **Open workplace:** a large room with small cubicles preferred
- ◆ **40-hour week:** No Overtime in two consecutive weeks
- ◆ **Just rules:** Team has its own but changeable rules for all to follow

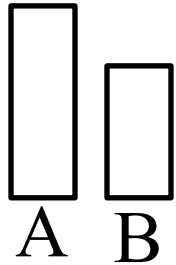
Popular PM elements

- ◆ Estimate the *story point* for each user story
 - Allocate extra time for research on a user story with high uncertainty
- ◆ Pick a set of user stories into the next iteration in XP (equivalent to the next sprint in Scrum)
- ◆ Within a cycle (an XP iteration or a Scrum sprint), conduct (daily) standing meetings
 - A super short meeting (15 mins in total)
 - [3 mins] Every team member reports
 - (1) what was done yesterday,
 - (2) what will be done today, and
 - (3) what blockers are encountered. For this item, other teammates may help resolve it.
- ◆ Measure *project velocity*
- ◆ At the end of each cycle, conduct a review (a demo to users about the deliverable achieved in the cycle) and a self-reflection on the cycle

User story

- ◆ A user story is in the following format:
 - As a <role>, I want an <action>, so that I can achieve a <goal>.
 - role is what we can identify from Requirements Engineering
 - action should be tangible (actionable)
 - goal should be measurable and quantifiable
- E.g., (for AIMS): As a CityU Student, I want to download a testimonial for my student status by submitting a request in AIMS so that I can apply for my student visa extension through the HKSAR Immigration Department.
- ◆ A user story usually has more information such as use cases, diagram sketch, user notes, data, reports from existing systems to clarify the context and scope of the user story.

Relative Story Point Estimation

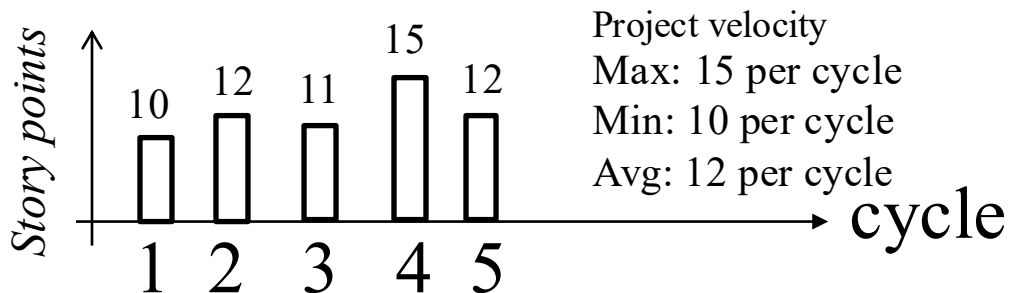


- ◆ It is difficult to estimate whether bar A is long and how long bar A is.
- ◆ However, it would be easier to say that A is longer than B.

- ◆ A story point is a value.
 - E.g., Fibonacci sequence: 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, ...
 - E.g., T-shirt size: XXS, XS, S, M, L, XL, XXL
- ◆ Pick one or more user stories as anchors. The whole team agreed on a story point for each such user story to indicate the number of expected efforts to implement the user stories, taking into account uncertainty and implementation complexity.
 - E.g., if B is agreed to as 3 story points, and the team has the impression that A is bigger than B but not very much, A can be assigned 5 story points.

Project Velocity

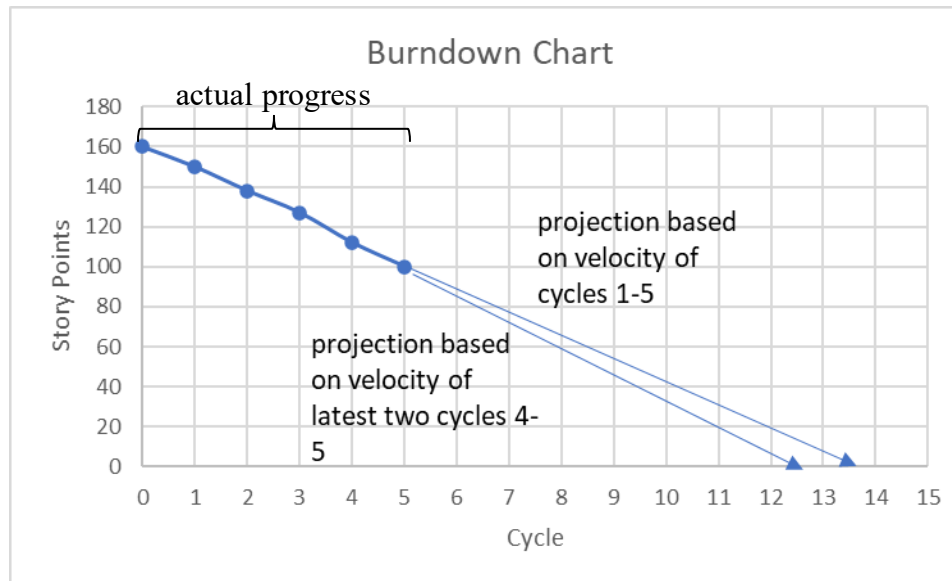
- ◆ A user story is implemented in a cycle if it passes the cycle review (by users and product owner)
- ◆ An agile project is organized as a series of cycles (XP iterations or Scrum sprints).
- ◆ Each cycle implements a set of user stories with assigned story points.
- ◆ If a user story is too large to fit into a cycle, the user story should be broken down into multiple user stories.
- ◆ The total story points delivered by the cycle are plotted.



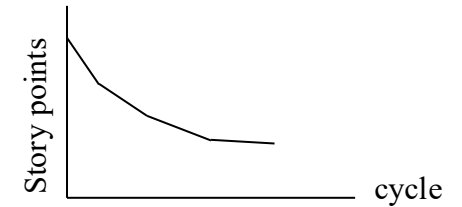
If the outstanding number of story points is 100, then the project needs $\text{ceiling}(100/12) = 9$ more cycles to complete

Burndown Chart

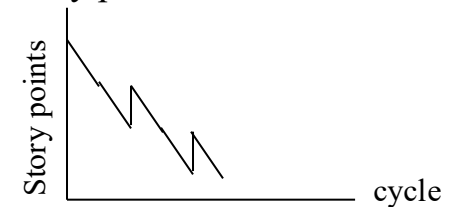
Cycle	Story Points
0	160
1	150
2	138
3	127
4	112
5	100



Problem! The project cannot make progress gradually



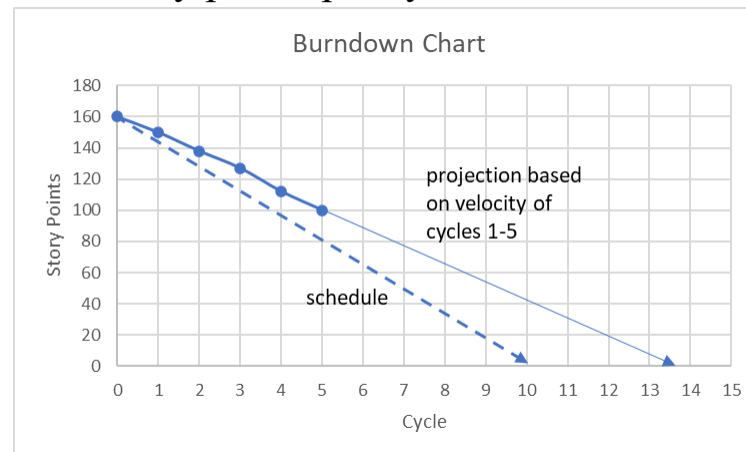
Oops! New user stories added to the project or original estimated story points are underestimated



Project velocity is the slope over a consecutive series of cycles.

Scheduled versus Actual

- ◆ When creating a release plan or a plan for a sprint, the project manager needs a schedule.
- ◆ Consider our course project (must deliver by the end of Week 12).
 - Suppose the project starts at Week 2, each cycle being 1 week. The last cycle is to wrap out the project and write a report, prepare a poster or videos, etc.
 - At the end of Week 7 (5 cycles completed), the project is delayed.
 - PM needs to take actions after each cycle: e.g., trim off some user stories and request the team to deliver more story points per cycle



Scrum

[http://en.wikipedia.org/wiki/Scrum_\(development\)](http://en.wikipedia.org/wiki/Scrum_(development))

- ◆ Is a kind of agile development process
- ◆ Arguably the most widely used development process nowadays
- ◆ driven by daily and periodic one-hour meetings
 - Go through the usage scenarios/stories to identify a slice of highest priority backlog tasks to be completed by the next iteration (Sprint) agreed by the customer
 - Together with customers, discuss the goal of the current Sprint, prioritize functions to be completed and divided into detailed tasks
 - Conduct periodic short planning and review meetings for tasks completed and not completed in time and revise the set of backlog tasks accordingly
- ◆ Customers may change their mind at any time
 - Accept that the requirements cannot be fully understood or defined,
 - Focus on maximizing the team's ability to deliver quickly and respond to the selected backlog tasks.

Watch this video: <https://tw.voicetube.com/videos/49552> (ignore the ads)

Scrum

Task board

Story	To Do		In Process	To Verify	Done
As a user, I... 8 points	Code the... SC 8	Test the... 8	Code the... SC 4	Test the... SC 6	Code the... SC 12
	Code the... 2	Code the... 8	Test the... SC 8		Test the... SC 8
	Test the... 8	Test the... 4			Test the... SC 8
As a user, I... 5 points	Code the... 8	Test the... 8	Code the... SC 8		Test the... SC 8
	Code the... 4	Code the... 6			Test the... SC 6

- ◆ A product backlog = user stories and development tasks due to the completion of some user stories (e.g., bugs found in a later cycle)
- ◆ Priority the product backlog
- ◆ Suppose a cycle = two weeks (so, our project has six sprints at most)
- ◆ A high-priority subset of the product backlog is treated as the current sprint backlog
 - The size of the sprint backlog is determined by the total number of story points that the team can be delivered within a cycle based on the history
- ◆ Through the standing meeting, each team member picks an item from the sprint backlog. The target is to deliver the user story in the cycle. (Visualize as a **task board**)
 - The tasks needed to deliver the story when it is started are all placed into the “to-do” column of the task board.
 - When a task is being worked on, it is moved to the “in-progress” column.
 - When the implementation of the task is completed, it is moved to the “to verify” column (which is usually merged with the “in-progress” column)
- ◆ At the end of a sprint, conduct a sprint review
 - Product owners/users verify whether the user story is delivered. If ok, then move the task to the “Done” column.
- ◆ Also, conduct a sprint retrospective (reflect on what has been done right to make progress and what to be improved to reduce or avoid blockers to improve productivity, etc)

Scrum Tools and Meeting

<https://www.mountangoatsoftware.com/agile/scrum/scrum-tools>

- ◆ **product backlog** is a list of features desired for a final product, the bugs to be removed, technical work to set up and maintain development environment and user site, and knowledge (e.g., learn to use a new framework) to acquire by the project.
- ◆ **release burndown chart** tracks progress on a project. The chart itself is updated after each sprint. Teams can measure progress in any unit they choose.
- ◆ **sprint backlog** is a list of tasks to complete during a sprint. It is updated once a day.
- ◆ **task board** is a sheet that every member of the team can use and add to over the course of a sprint, and is a visual representation of every task and what phase of completion it's in. Usually, task boards include columns for stories, to-dos, work in process, things needing verification and items finished. Some teams also include burndown charts, notes and tests. **Hang the board on a wall** or digitalize it.
- ◆ **Meeting**: The product owner shows up at the sprint planning meeting with the prioritized agile product backlog and describes the top items to the team. The team then determines which items they can complete during the coming sprint. The team then moves items from the product backlog to the sprint backlog. In doing so, they expand each Scrum product backlog item into one or more sprint backlog tasks

Scrum Tools

<https://www.mountangoatsoftware.com/agile/scrum/scrum-tools>

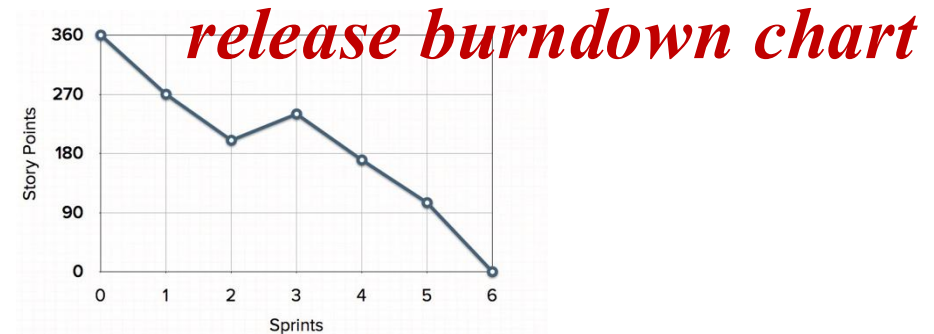
<https://www.mountangoatsoftware.com/agile/scrum/scrum-tools/product-backlog/example>

Profiles

- As a site member, I want to describe myself on my own page in a semi-structured way so that others can learn about me. That is, I can fill in predefined fields, but also have room for a free-text field or two. (It would be nice to let this free text be HTML or similar.)
- As a site member, I can fill out an application to become a Certified Scrum Practitioner so that I can earn that designation. [Note: Certified Scrum Practitioner was the initial name of what became known as Certified Scrum Professional.]
- As a Practitioner, I want my profile page to include additional details about me (i.e. some of the

task board

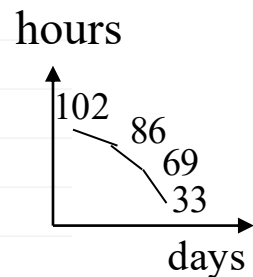
Story	To Do		In Process	To Verify	Done
As a user, I... 8 points	Code the... 9	Test the... 8	Code the... DC 4	Test the... SC 6	Code the... 8 Test the... SC 8 Test the... SC 6 Test the... SC 6
As a user, I... 5 points	Code the... 8	Test the... 8	Code the... DC 8		Test the... SC 8 Test the... SC 6 Test the... SC 6



sprint backlog

Hours to go to complete the task

USER STORY	TASKS	DAY 1	DAY 2	DAY 3	DAY 4	DAY 5	...
As a member, I can read profiles of other members so that I can find someone to date.	Code the...	8	4	8	0		
	Design the...	16	12	10	4		
	Meet with Mary about...	8	16	16	11		
	Design the UI	12	6	0	0		
	Automate test...	4	4	1	0		
As a member, I can update my billing information	Code the other...	8	8	8	8		
	Update security tests	6	6	4	0		
	Design a solution to...	12	6	0	0		
	Write a test plan	8	8	4	0		
	Automate tests...	12	12	10	6		
	Code the...	8	8	8	4		



Sum: 102 86 69 33

Scrum Roles

- ◆ *Scrum Master* is the team's coach and helps Scrum practitioners achieve their highest level of performance.
 - This role does not provide day-to-day direction to the team and does not assign tasks to individuals.
 - In many projects, this role is assumed by the project manager.
- ◆ *Product owner* is responsible for prioritizing the backlog during Scrum development.
- ◆ *Scrum development team* as a whole.

Comparison among Agile Methods

Extreme Programming

Method name	Key points	Special features	Identified shortcomings
	the method development.	“visionary” and “advisor”.	method.
XP	Customer driven development, small teams, daily builds	Refactoring – the ongoing redesign of the system to improve its performance and responsiveness to change.	While individual practices are suitable for many situations, overall view & management practices are given less attention.
FDD	Five-step process, object-oriented component (i.e., feature) based development. Very short iterations: from hours to 2 weeks.	Method simplicity, design and implement the system by features, object modeling.	FDD focuses only on design and implementation. Needs other supporting approaches.
OSS	Volunteer based, distributed development, often the problem domain is more of a challenge than a commercial undertaking.	Licensing practice; source code freely available to all parties.	OSS is not a method itself; ability to transform the OSS community principles to commercial software development.
PP	Emphasis on pragmatism, theory of programming is of less importance, high level of automation in all aspects of	Concrete and empirically validated tips and hints, i.e., a pragmatic approach to software	PP focuses on important individual practices. However, it is not a method through which a system can be

Method name	Key points	Special features	Identified shortcomings
	programming.	development.	developed.
RUP	Complete SW development model including tool support. Activity driven role assignment.	Business modeling, tool family support.	RUP has no limitations in the scope of use. A description how to tailor, in specific, to changing needs is missing.
Scrum	Independent, small, self-organizing development teams, 30-day release cycles.	Enforce a paradigm shift from the “defined and repeatable” to the “new product development view of Scrum.”	While Scrum details in specific how to manage the 30-day release cycle, the integration and acceptance tests are not detailed.

Scrum

Home Reading

- ◆ Read the following section of this reference:
<https://arxiv.org/ftp/arxiv/papers/1709/1709.08439.pdf>
 - Section 3.2 Scrum
- ◆ Read the differences in Scrum Master and Product Owner in Scrum
 - <https://www.yodiz.com/blog/scrum-master-vs-product-owner-differences-in-skills-duties-and-responsibilities-agile-methodology/>

References

1. João M. Fernandes and Sónia M. Sousa. 2010. PlayScrum - A Card Game to Learn the Scrum Agile Method. In Proceedings of the 2010 Second International Conference on Games and Virtual Worlds for Serious Applications (VS-GAMES '10). IEEE Computer Society, Washington, DC, USA, 52-59. DOI: <https://doi.org/10.1109/VS-GAMES.2010.24>
 - Include a concise summary of Scrum
2. Videos on YouTube
 - https://www.youtube.com/results?search_query=scrum